

**Instituto Politécnico Nacional**
**Escuela Superior de Cómputo**




Ingeniería en Sistemas Computacionales · Plan 2020

Unidad de aprendizaje: **Teoría de la Computación**

# Práctica 2

## Autómatas Finitos No Deterministas (AFND) y Simulación de AFD con Interfaz Gráfica

**Alumno:** Rico Espluga Juan Diego
**Boleta:** 2025630474
**Grupo:** 4CV4
**Docente:** Gabriel Hurtado Avilés
**Fecha de entrega:** 6 de octubre de 2026

# Índice

**1. Objetivo** **3**

**2. Ejercicio 1. Entorno de trabajo y flujo de ramas** **3**

2.1. Lo que se reutiliza . . . 3

2.2. Una rama por funcionalidad . . . 3

2.3. Integración continua . . . 4

**3. Ejercicio 2. AFND y AFND-$\lambda$ en JFLAP** **6**

3.1. Clasificación y cobertura . . . 6

3.2. Diseño y validación . . . 6

3.3. Transiciones $\lambda$ . . . 7

3.4. Conversiones y análisis comparativo . . . 10

**4. Ejercicio 3. Simulador de autómatas con interfaz gráfica** **11**

4.1. Organización del código . . . 11

4.2. Definición interactiva . . . 12

4.3. Importación desde archivos . . . 13

4.4. Simulación y validación . . . 14

4.5. Exportación y operaciones sobre lenguajes . . . 15

4.6. Pruebas . . . 15

**5. Ejercicio 4. Investigación y estado del arte** **16**

5.1. Investigación . . . 16

5.1.1. El no determinismo . . . 16

5.1.2. La $\lambda$-clausura . . . 16

5.1.3. Construcción de subconjuntos y equivalencia AFD–AFN . . . 18

5.1.4. El costo de la conversión . . . 19

5.1.5. Teorema de Kleene . . . . . 19
5.1.6. Minimización: Myhill-Nerode y Hopcroft . . . . . 19
5.1.7. Qué significa que el no determinismo no aumente el poder . . . . . 20
5.2. Artículo asignado: Rabin y Scott (1959) . . . . . 20
5.3. Dos artículos localizados . . . . . 21
5.3.1. Saarikivi, Veanes, Wan y Xu (2019) . . . . . 21
5.3.2. Moseley et al. (2023) . . . . . 22

**6. Ejercicio 5. Del artículo al autómata** **22**
6.1. La aplicación elegida . . . . . 22
6.2. Definición formal . . . . . 23
6.3. Cadenas de prueba . . . . . 26
6.4. La abstracción . . . . . 26

**7. Propuesta de trabajo** **27**

**8. Conclusiones** **28**

**A. Anexo A. Clasificación de los ejercicios** **32**

**B. Anexo B. Cerraduras $\lambda$** **39**

**C. Anexo C. Conversiones** **42**

# 1. Objetivo

Profundizar en los autómatas finitos no deterministas y sus conversiones mediante JFLAP, desarrollar una aplicación gráfica que defina, importe y simule autómatas en varios formatos, y trasladar a un modelo formal propio una aplicación descrita en la literatura.

La práctica tiene cinco ejercicios y continúa la forma de trabajo de la Práctica 1. Las fichas completas de los 140 autómatas y el recorrido de cada cadena de prueba están en `docs/fichas.md` y `docs/recorridos/`; este documento incluye la clasificación, las cerraduras $\lambda$ y las conversiones como anexos.

# 2. Ejercicio 1. Entorno de trabajo y flujo de ramas

## 2.1. Lo que se reutiliza

El repositorio `practica2-tc` parte de la Práctica 1: la carpeta `entorno/` (el `Dockerfile` parametrizado y el `compose.yml` con los servicios `py311`, `py312` y `py313`), `requirements.txt` y `pytest.ini`. Sólo cambian los nombres de los contenedores (`tc_p2_py311`, …) para que puedan convivir con los de la práctica anterior.

El módulo `src/nucleo/lenguajes.py`, junto con `errores.py` y sus pruebas (`tests/test_lenguajes.py`), se copió **sin modificaciones** de la Práctica 1, como pide el enunciado; el `README` lo indica y enlaza el repositorio `practica1-tc`. La interfaz conserva las secciones *Palabras*, *Cerraduras* y *Lenguajes* y agrega las de esta práctica.

## 2.2. Una rama por funcionalidad

Ninguna confirmación se hizo directamente sobre `main`: cada funcionalidad se desarrolló en su rama y se integró con un Pull Request que describe el cambio y muestra el resultado de las pruebas. La rama `main` se protegió en *Settings* → *Branches* para que GitHub rechace los `push` directos.

**Tabla 1.** Ramas fusionadas.

| Rama | Qué integra |
|---|---|
| `feature/entorno-ci` | Entorno copiado de la Práctica 1 y flujo de GitHub Actions. |
| `feature/nucleo-automatas` | `automata.py`: `Automata`, `AFNLambda`, `AFN`, `AFD`, `Simulacion` y `Recorrido`, con sus pruebas. |
| `feature/formatos` | `formatos.py`: serializadores `.jff`, `.json` y `.xml` y pruebas de ida y vuelta. |
| `feature/editor` | Sección *Editor*: quíntupla, tabla de transiciones y vista previa. |
| `feature/biblioteca-simulador` | Secciones *Biblioteca* y *Simulador*, con la ejecución paso a paso. |
| `feature/fantasma-parcial` | Ejercicio 5: el modelo del fantasma, sus archivos y sus pruebas. |
| `feature/automatas-listas` | Los 40 autómatas de las Listas 2 y 3 en tres formatos y su prueba. |

## 2.3. Integración continua

`.github/workflows/pruebas.yml` es el flujo del Anexo 2: en cada `push` y en cada Pull Request, la matriz crea tres trabajos independientes, uno por versión de Python, y `fail-fast: false` evita que el fallo de una versión cancele las otras (GitHub Inc., 2026). La suite tiene 404 pruebas aprobadas y 2 omitidas (las de secciones de prácticas posteriores):

**Tabla 2.** Composición de la suite.

| Archivo | Pruebas | Qué comprueba |
|---|---|---|
| `test_lenguajes.py` | 34 | Operaciones de la Práctica 1 (sin cambios). |
| `test_automata.py` | 34 | Clasificación, AFD, AFN, cerradura $\lambda$, simulación paso a paso y validación de la quíntupla. |
| `test_formatos.py` | 24 | Ida y vuelta en los tres formatos, lectura del esquema del Anexo 3 y archivos dañados. |
| `test_listas.py` | 281 | Cada autómata de las listas acepta sus válidas, rechaza sus inválidas y se lee igual en los tres formatos. |
| `test_aplicacion.py` | 20 | Modelo del Ejercicio 5 y sus quince cadenas. |
| `test_interfaz.py` | 13 | Cada sección se construye y responde sin abrir una ventana. |

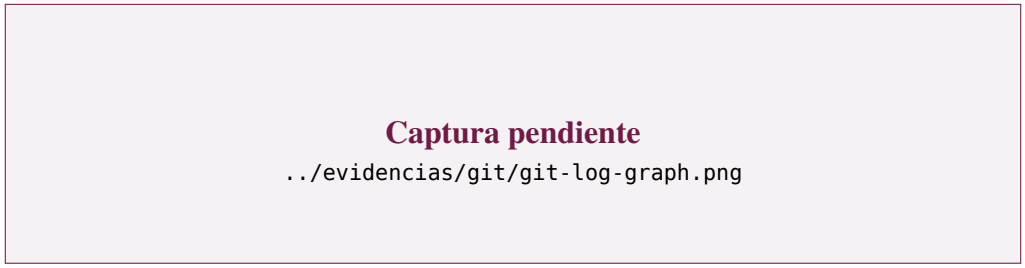



**Figura 1.** `git log --oneline --graph --all` con las fusiones de las ramas.

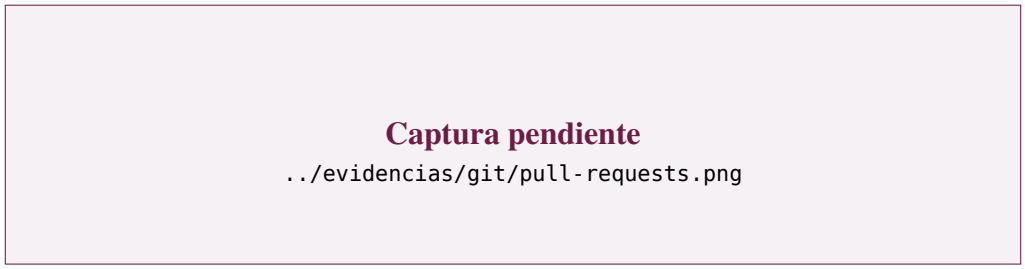



**Figura 2.** Pull Requests cerrados.

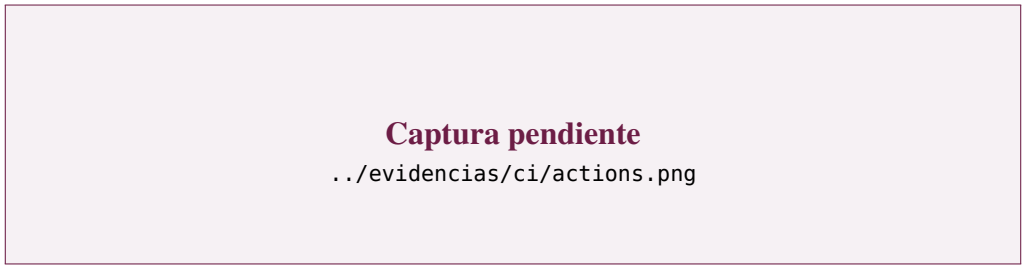



**Figura 3.** GitHub Actions: los tres trabajos de la matriz aprobados.

# 3. Ejercicio 2. AFND y AFND-$\lambda$ en JFLAP

## 3.1. Clasificación y cobertura

Con los ejercicios de la Práctica 1 y los que se añaden aquí, las Listas 2 y 3 quedan implementadas por completo: **140 autómatas**. Cada uno se clasificó por la forma de su función de transición, no por cómo se llamaba en el enunciado:

- **AFD** si cada par (estado, símbolo) tiene a lo sumo un destino y no hay transiciones $\lambda$;
- **AFND** si algún par tiene dos o más destinos, pero no hay transiciones $\lambda$;
- **AFND-$\lambda$** si hay al menos una transición $\lambda$.

**Tabla 3.** Clasificación de los 140 autómatas.

| Sección | Lista 2 | Lista 3 | Total | Carpeta |
|---|---|---|---|---|
| AFD | 61 | 31 | 92 | `automatas/afd/` |
| AFND | 0 | 25 | 25 | `automatas/afnd/` |
| AFND-$\lambda$ | 0 | 23 | 23 | `automatas/afnd-lambda/` |

Cinco ejercicios no producen autómata: 116 (y su versión AFN) describen un lenguaje que no es regular, 143 depende de figuras que no aparecen en la lista, 191 es un cálculo sobre un AFN dado y 201–235 son afirmaciones de verdadero o falso. El Anexo A justifica en una línea la sección de cada ejercicio. El patrón que se repite es claro: el no determinismo aparece cuando el enunciado obliga a *adivinar* una posición («termina en…», «el antepenúltimo símbolo es…») o a elegir entre dos condiciones; las transiciones $\lambda$, cuando el lenguaje se arma pegando piezas con unión, concatenación o estrella, como en las expresiones regulares 236–253.

## 3.2. Diseño y validación

Los AFND se diseñaron aprovechando justamente esa capacidad de adivinar: para «contiene 00101 o 00110» (189) basta un lazo inicial que consume cualquier cosa y dos ramas que leen cada patrón, sin calcular el autómata de búsqueda que necesitaría un AFD. Cada autómata tiene en `docs/fichas.md` su ficha con el diseño, la tabla de transiciones y **cinco cadenas válidas y cinco**

**inválidas** de longitudes crecientes, más los casos frontera; cuando el lenguaje o su complemento tienen menos de cinco palabras, se listan todas. El resultado esperado se obtuvo de la definición del lenguaje y el obtenido, del simulador del Ejercicio 3; además, cada autómata se comparó con la definición sobre todas las cadenas de longitud $\leq 14$ ($\leq 9$ con tres símbolos).

El **recorrido completo** de cada cadena está en `docs/recorridos/<ejercicio>.md`. Por cada símbolo se muestran los estados activos antes y después, las transiciones usadas y si hubo bifurcación. Por ejemplo, en el AFN 192 con la palabra `1010`:

```
Inicio: {A}
Lee 1: {A} ⟶ {B, C}        [A→B, A→C]                (bifurcación)
Lee 0: {B, C} ⟶ {A, B, C}  [B→A, B→B, C→A, C→C]     (bifurcación)
Lee 1: {A, B, C} ⟶ {B, C}  [A→B, A→C]
Lee 0: {B, C} ⟶ {A, B, C}  [B→A, B→B, C→A, C→C]     (bifurcación)
Aceptada: queda {C} ∈ F.
```

En JFLAP se usó *Input → Step with Closure*, que muestra todas las configuraciones vivas a la vez; las capturas corresponden a cinco autómatas de distinta clase.

### 3.3. Transiciones $\lambda$

En los 23 AFND-$\lambda$ las transiciones $\lambda$ cumplen tres papeles: **unir** dos subautómatas sin duplicar estados (de la unión r + s), **encadenar** el final de una pieza con el inicio de la siguiente (concatenación) y **regresar** o **saltar** en una cerradura (estrella). Están colocadas exactamente en esos puntos de unión; en ningún caso consumen un símbolo, sólo cambian de pieza.

La **$\lambda$-clausura** de q, $\lambda$-cl(q), es el conjunto de estados alcanzables desde q usando sólo transiciones $\lambda$, incluido q. Se calcula con un recorrido en profundidad sobre las aristas $\lambda$. Su papel en el reconocimiento es doble: la lectura no empieza en $\{q_0\}$ sino en $\lambda$-cl($q_0$), y después de mover el conjunto de activos con un símbolo hay que volver a cerrarlo. En el AFND-$\lambda$ 197, por ejemplo, $\lambda$-cl(A) = {A, B, C, D} contiene al estado final D, de modo que $\lambda$ se acepta aunque el estado inicial no sea final. El Anexo B lista la cerradura de cada estado de los 23 AFND-$\lambda$.

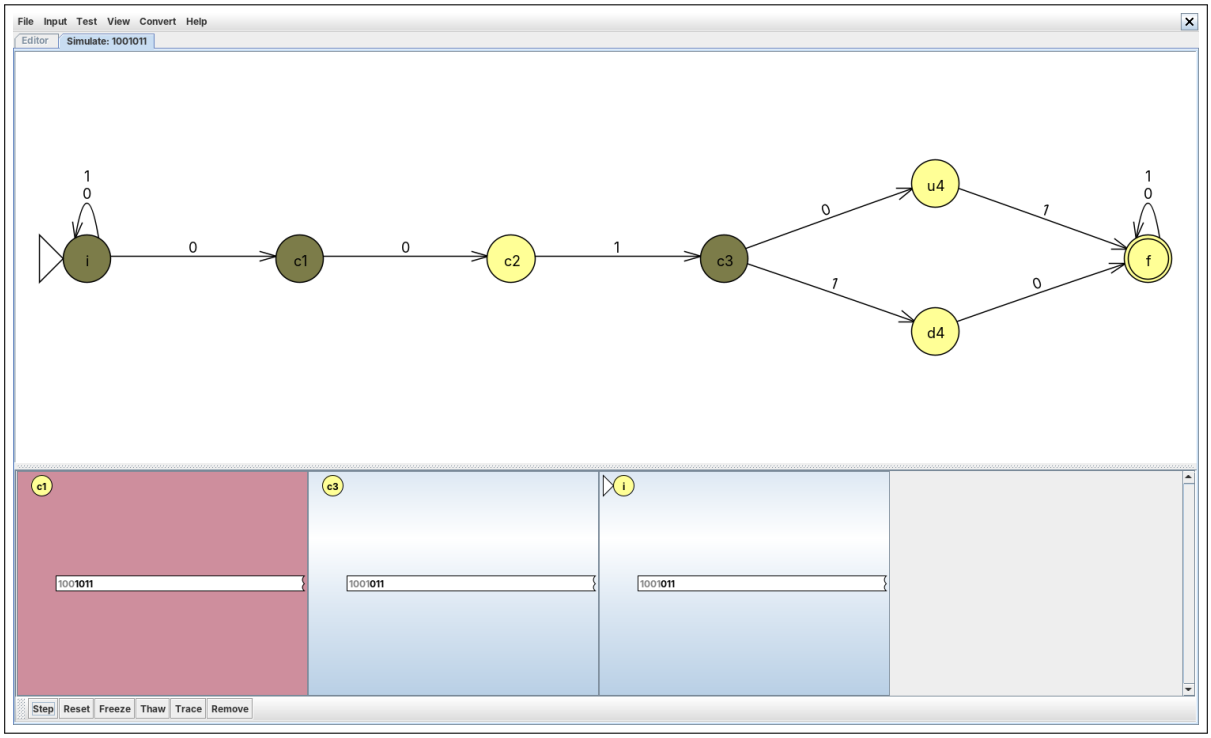



**Figura 4.** JFLAP, AFND 189 con `1001011`: las configuraciones de las dos ramas en paralelo.

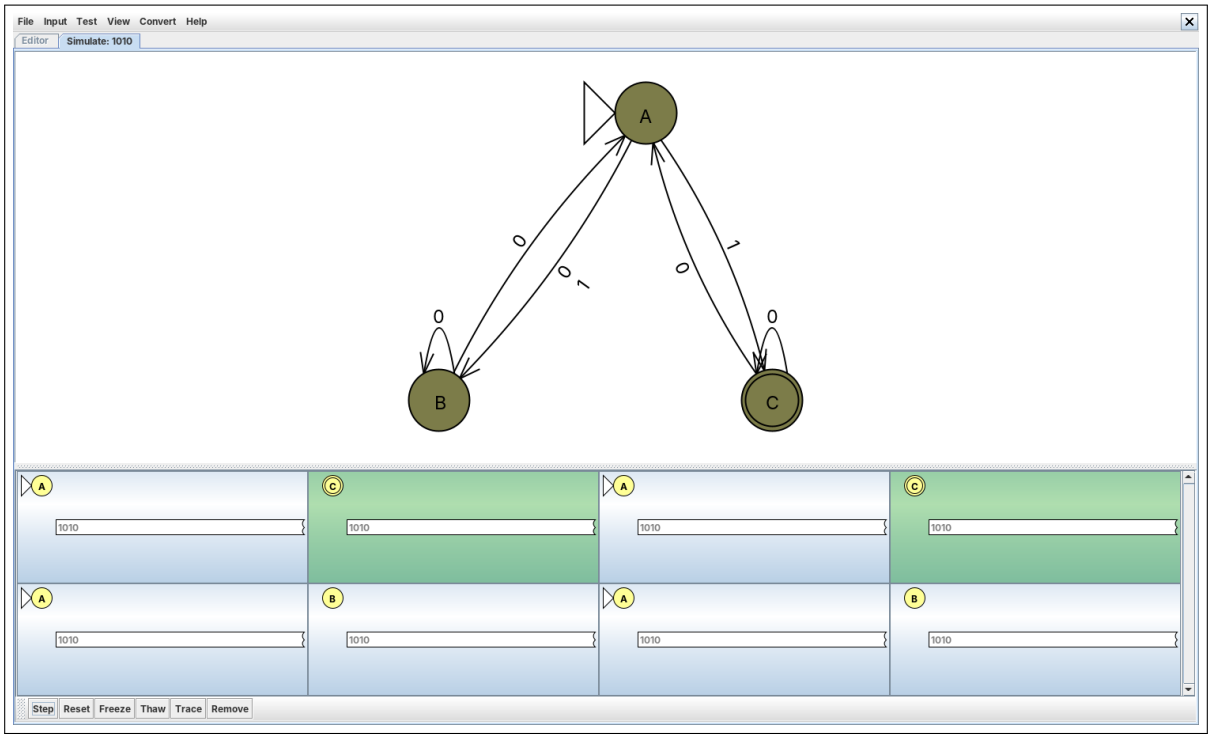



**Figura 5.** JFLAP, AFND 192 con `1010`.

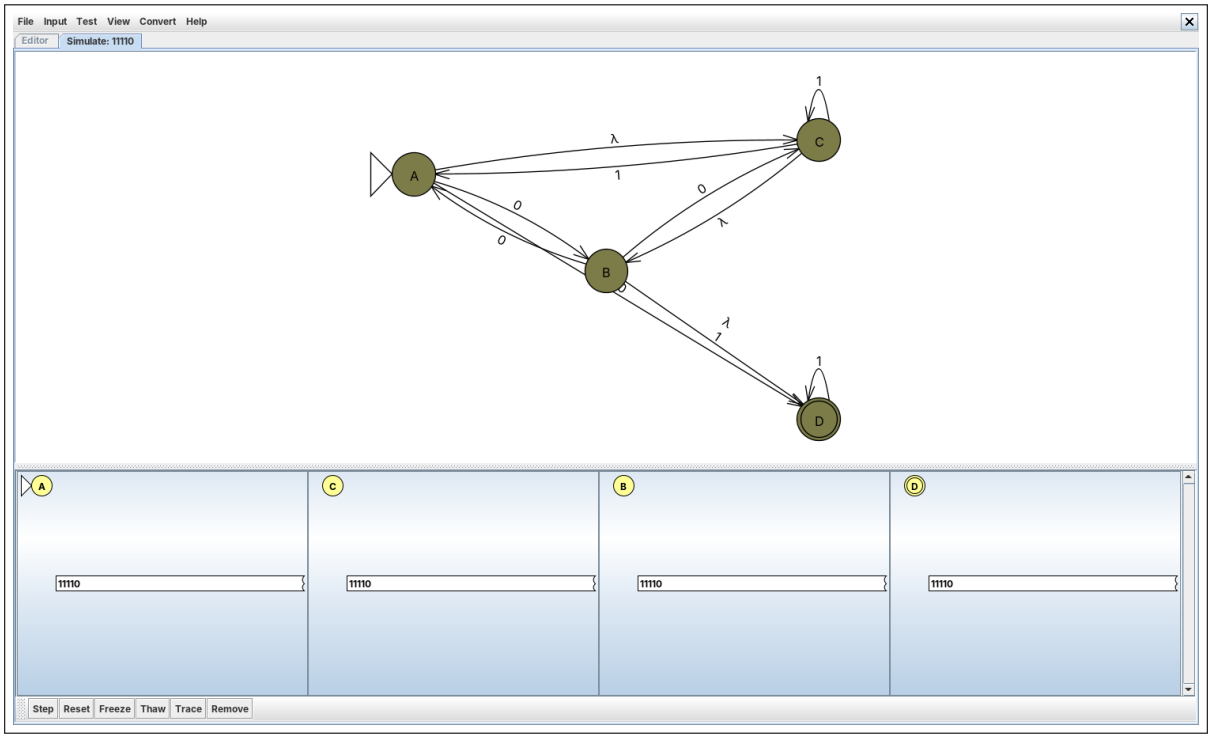



**Figura 6.** JFLAP, AFND-$\lambda$ 197: la cerradura del estado inicial ya contiene el estado final.

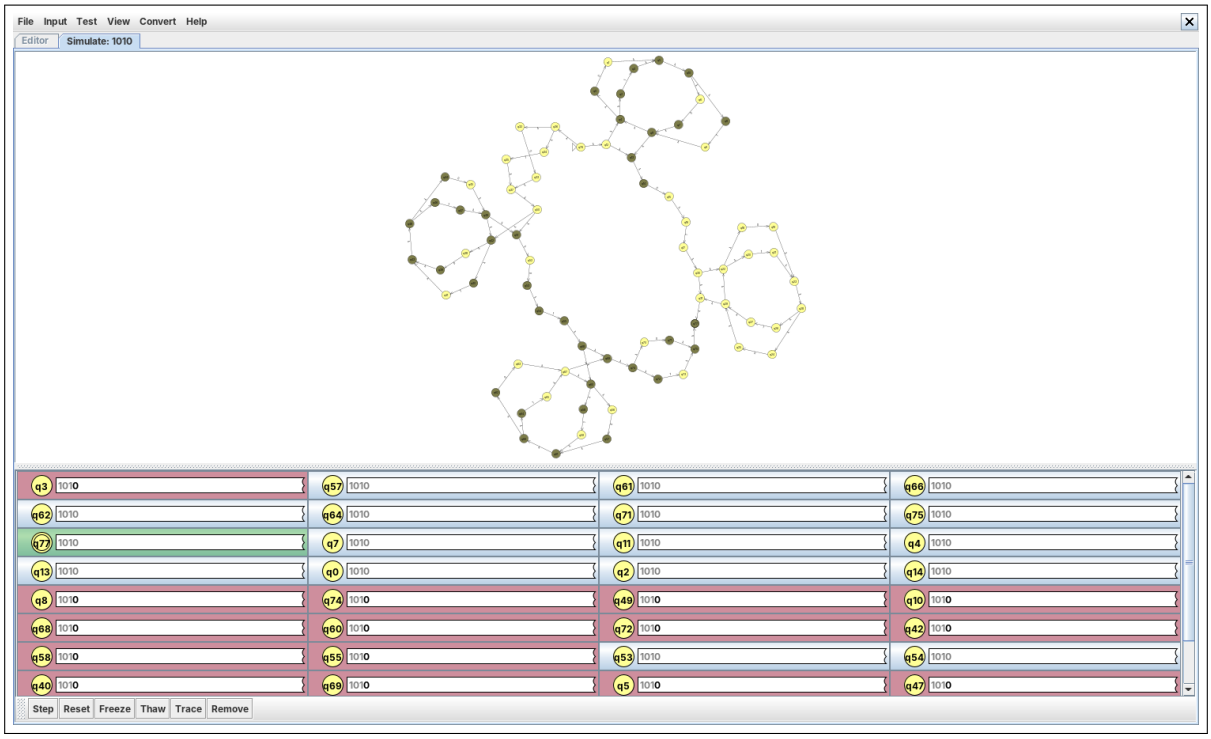



**Figura 7.** JFLAP, AFND-$\lambda$ 240 (longitud par con 01) con `1010`.

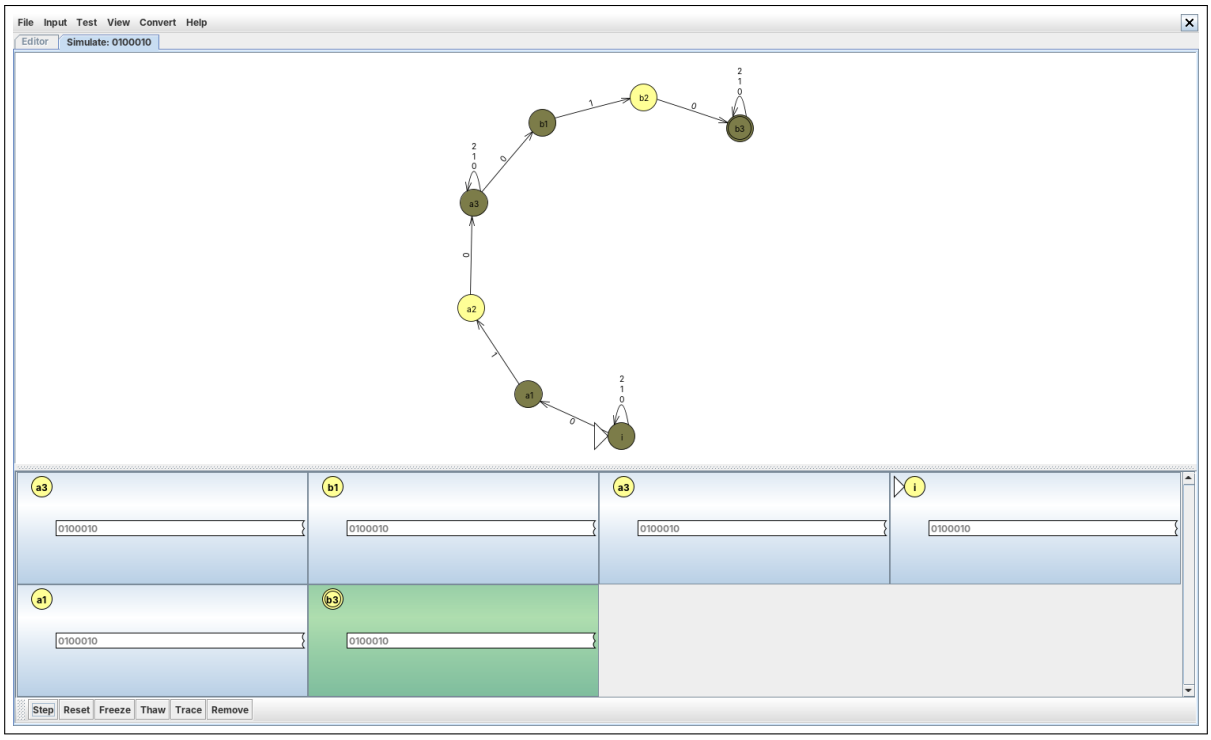



**Figura 8.** JFLAP, AFND del problema 112 con `0100010`.

### 3.4. Conversiones y análisis comparativo

Cada AFND se convirtió en AFD con *Convert* → *Convert to DFA* de JFLAP, y cada AFND-$\lambda$ primero en AFND (eliminando $\lambda$) y después en AFD; los resultados están en `automatas/conversiones/` y se comprobaron con el simulador. El Anexo C da los tamaños de los 48 casos.

**Tabla 4.** Tamaño antes y después de la conversión.

| Ejercicio | Clase | Estados del AFND | Estados del AFD | AFD mínimo | Cota $2^n$ |
|---|---|---|---|---|---|
| 192 | AFND | 3 | 5 | 5 | 8 |
| 193 | AFND | 4 | 7 | 6 | 16 |
| 195 | AFND | 5 | 12 | 7 | 32 |
| 189 | AFND | 7 | 12 | 7 | 128 |
| 190 | AFND | 26 | 32 | 19 | 67 108 864 |
| 199 | AFND-$\lambda$ | 5 | 4 | 2 | 32 |
| 200 | AFND-$\lambda$ | 6 | 6 | 2 | 64 |

**Número de estados.** El AFD nunca superó el 62.5 % de la cota (192: 5 de 8), y en los casos grandes queda muy lejos: en 190, 32 subconjuntos de $2^{26}$ posibles. La construcción sólo genera los subconjuntos alcanzables desde $\lambda$-cl($q_0$), y en un diseño típico los estados del AFND se activan en grupos correlacionados (si estoy en el tercer símbolo de un patrón, también estoy en el lazo inicial), de modo que la mayoría de los subconjuntos nunca aparecen.

**Complejidad de las transiciones.** El AFND tiene menos transiciones y cada una se lee en el enunciado («después de 0 puede empezar el patrón»); el AFD tiene exactamente una por par (estado, símbolo) y absorbe en ellas la lógica de los retrocesos.

**Facilidad de diseño e interpretación.** El AFND es más fácil de diseñar y de explicar; los estados del AFD son conjuntos y su significado hay que deducirlo.

**Eficiencia.** El AFD reconoce en tiempo O(|ω|) con una consulta por símbolo; simular el AFND cuesta O(|ω|·|Q|²) en el peor caso, porque en cada paso se mueve un conjunto de hasta |Q| estados. Convertir cuesta, en el peor caso, $2^n$ estados: conviene cuando un mismo autómata se usará con muchas cadenas.

**Una familia en la que sí se alcanza la cota.** Sea $L_n$ el lenguaje de las cadenas sobre {0, 1} cuyo n-ésimo símbolo desde el final es 1. Un AFND con n + 1 estados lo reconoce (adivina dónde está ese símbolo y cuenta n − 1 más), pero todo AFD necesita al menos $2^n$ estados: dos prefijos distintos de longitud n difieren en alguna posición i, y añadirles i − 1 símbolos los separa, así que todos los $2^n$ prefijos deben llevar a estados distintos (Hopcroft et al., 2007, sec. 2.3.6).

## 4. Ejercicio 3. Simulador de autómatas con interfaz gráfica

### 4.1. Organización del código

El núcleo (`src/nucleo/`) no importa Flet; la interfaz (`src/interfaz/`) sólo lo invoca.

**Tabla 5.** Módulos del núcleo.

| Módulo | Clases | Responsabilidad |
|---|---|---|
| `lenguajes.py` | `Alfabeto`, `Palabra`, `Lenguaje`, `Enumerador` | Práctica 1, sin cambios. |
| `automata.py` | `Automata` → `AFNLambda` → `AFN` → `AFD` | Quíntupla, validación, cerradura $\lambda$, $\delta$, tabla de transiciones. |
| | `Arista`, `Paso`, `Recorrido`, `Simulacion` | Una transición; un símbolo leído; el resultado completo; la ejecución interactiva. |
| | `Clasificador`, `Disposicion` | Deducir el tipo y explicarlo; coordenadas para dibujar y para JFLAP. |
| `formatos.py` | `Serializador` → `SerializadorJFF`, `SerializadorJSON`, `SerializadorXML`; `Archivos` | Escribir y leer cada formato; elegir el formato por la extensión. |

La jerarquía refleja la teoría: un AFD es un AFN en el que cada par tiene a lo sumo un destino, y un AFN es un AFN-$\lambda$ sin transiciones $\lambda$. Cada subclase sólo **añade restricciones** en `_validar` (un `AFD` rechaza dos destinos para el mismo par; un `AFN`, las transiciones $\lambda$) y atajos (`AFN.cierre` devuelve el mismo conjunto, `AFD.siguiente` el único destino). El tipo no se declara: `Automata.crear` construye la quíntupla general y `Clasificador.clase_de` elige la clase más específica que la admite.

### 4.2. Definición interactiva

La sección **Editor** se divide en tres tarjetas numeradas. En *1 · Componentes* se escriben $\Sigma$ y Q separados por comas, se elige $q_0$ en una lista y se marcan los estados de aceptación tocando sus pastillas; un interruptor agrega la columna $\lambda$. En *2 · Función de transición* aparece una rejilla con una fila por estado y una columna por símbolo; cada celda recibe los destinos separados por comas y una celda vacía significa «sin transición». La rejilla se rehace sola cuando cambian $\Sigma$ o Q y conserva lo que ya estaba escrito. En *3 · Vista previa* se ven, mientras se escribe, el diagrama,

la tabla completa y una etiqueta con el tipo deducido y su motivo («Hay elección: $\delta$(q0, a) = {q0, q1}»); un error (un destino que no existe, un inicial fuera de Q) aparece en rojo en el mismo lugar. *Editar el autómata activo* vuelve a llenar las tres tarjetas para corregir una definición.

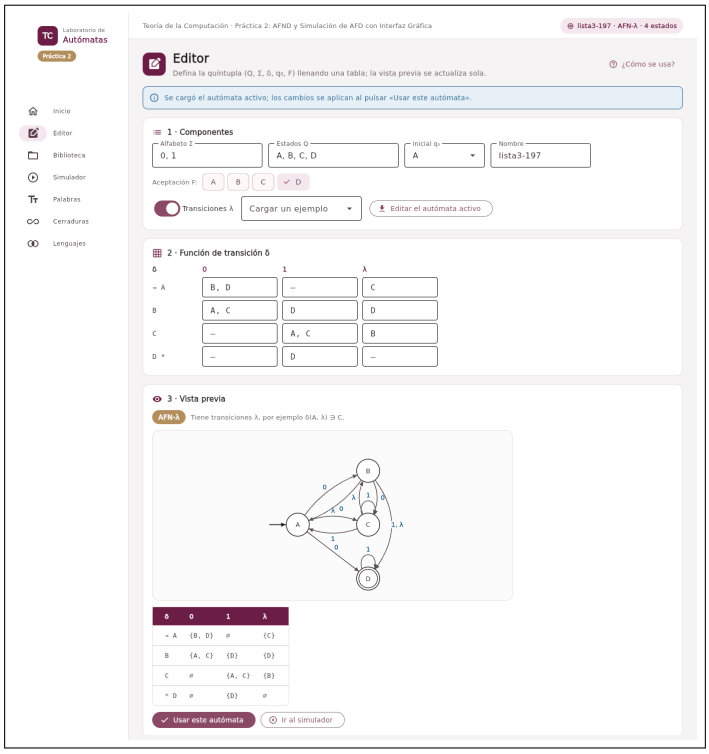



**Figura 9.** Editor: AFN-$\lambda$ de ejemplo con su tabla, su diagrama y el tipo deducido.

### 4.3. Importación desde archivos

La sección **Biblioteca** lista los archivos de `automatas/` con un buscador; al tocar uno se muestran su quíntupla (Q, $\Sigma$, $q_0$, F y número de transiciones), su diagrama y su tabla, sin cambiar el autómata activo hasta pulsar *Usar*. También acepta una ruta cualquiera.

**Esquemas.** El `.jff` es el de JFLAP 7.1. Para `.json` y `.xml` se definió un esquema propio que guarda la quíntupla completa, el tipo y las coordenadas:

```
{"formato": "automata-tc", "version": 1, "nombre": "anexo3", "tipo": "AFN-λ",
 "alfabeto": ["a", "b"], "estados": ["q0", "q1"], "inicial": "q0", "aceptacion":
 ↪  ["q1"],
 "delta": {"q0": {"a": ["q1"], "b": ["q0"], "": ["q1"]}, "q1": {}},
```

```
 "posiciones": {"q0": [100.0, 100.0], "q1": [270.0, 100.0]}}
```

```
<automata formato="automata-tc" version="1" nombre="anexo3" tipo="AFN-λ">
  <alfabeto><simbolo>a</simbolo><simbolo>b</simbolo></alfabeto>
  <estados>
    <estado id="q0" inicial="si" aceptacion="no" x="100.0" y="100.0" />
    <estado id="q1" inicial="no" aceptacion="si" x="270.0" y="100.0" />
  </estados>
  <delta>
    <desde estado="q0">
      <con simbolo="a"><hacia>q1</hacia></con>
      <con simbolo="b"><hacia>q0</hacia></con>
      <con simbolo=""><hacia>q1</hacia></con>
    </desde>
  </delta>
</automata>
```

Ambos agrupan $\delta$ por estado y por símbolo, así que un par (q, a) aparece una sola vez con todos sus destinos. **Convención para $\lambda$**: en los tres formatos la transición $\lambda$ se escribe con el símbolo vacío (`<read/>`, `""` y `simbolo=""`); al leer se aceptan también «$\lambda$», «ε», «lambda» y «eps». Para leer archivos de otras personas, los lectores aceptan además el esquema de ejemplo del Anexo 3 del enunciado (una lista de transiciones con `desde`, `lee` y `hacia`).

**Interpretación de la quíntupla.** Al leer se verifica que Q no sea vacío ni tenga repetidos, que $\Sigma$ no contenga $\lambda$, que el inicial y los finales estén en Q y que cada transición use estados de Q y símbolos de $\Sigma$. Un archivo que se declara `AFD` pero no es determinista se rechaza indicando los pares que fallan. El `.jff` no guarda $\Sigma$: se infiere de los símbolos de las transiciones.

### 4.4. Simulación y validación

En **Simulador** se escribe la palabra (vacía o «$\lambda$» para la palabra vacía). *Validar* la procesa completa; *Paso a paso* prepara una cinta con un recuadro por símbolo y *Siguiente* lee uno a la vez: el símbolo actual se resalta en la cinta, los estados activos en el diagrama y en la tabla, y las transiciones que se acaban de usar se dibujan en guinda. La traza numerada muestra, por paso, el conjunto de activos antes, el conjunto al que se llega y, si cambió, su cierre $\lambda$. Al terminar aparece

un recuadro verde (ACEPTADA) o rojo (RECHAZADA) con el motivo: qué estado final quedó activo, o que no quedó ninguno.

**Decisión de diseño: seguir todos los caminos.** Para los no deterministas se eligió simular el conjunto de estados activos en lugar de convertir a AFD. En cada paso se calcula S ← $\lambda$-cl($\bigcup_{q \in S} \delta(q, a)$).

**Tabla 6.** Costo de las dos formas de simular un AFN.

| Alternativa | Costo de preparación | Costo por palabra | Ventaja |
|---|---|---|---|
| Conjuntos de activos | Cerraduras: $O(\|Q\|^2)$ una vez | $O(\|\omega\| \cdot \|Q\|^2)$ | Muestra las bifurcaciones y la cerradura tal como las define la teoría. |
| Convertir a AFD | Hasta 2^\|Q\| estados y $\|\Sigma\| \cdot 2^{\|Q\|}$ transiciones | $O(\|\omega\|)$ | Conviene si el mismo autómata procesará muchísimas palabras. |

Se eligió la primera porque el objetivo del simulador es mostrar el comportamiento del autómata que se diseñó, no el de otro equivalente, y porque nunca paga el costo exponencial; la conversión queda para la Práctica 3.

### 4.5. Exportación y operaciones sobre lenguajes

La tarjeta *Guardar* de **Biblioteca** escribe el autómata activo en `.jff`, `.json`, `.xml` o en los tres a la vez, en la carpeta `salidas/`. Las operaciones de la Práctica 1 (subcadenas, prefijos, sufijos, $\Sigma^*$ y $\Sigma^+$) siguen en las secciones **Palabras**, **Cerraduras** y **Lenguajes** de la misma ventana.

### 4.6. Pruebas

**Tabla 7.** Requisitos mínimos de la suite.

| Requisito | Prueba |
|---|---|
| Exportar e importar en los tres formatos son operaciones inversas | `test_ida_y_vuelta` (4 autómatas $\times$ 3 formatos): leer lo escrito da la misma quíntupla y volver a escribirlo da el mismo texto. |

| Requisito | Prueba |
|---|---|
| Aceptación y rechazo en un AFD | `test_afd_acepta`, `test_afd_rechaza`, `test_afd_parcial_y_completado`. |
| AFND-$\lambda$ que requiere la cerradura | `test_afn_lambda_requiere_la_cerradura`: en un AFN-$\lambda$ para a*b*, «b» sólo se acepta porque q1 $\in$ $\lambda$-cl(q0). |

Las 404 pruebas se aprueban en los tres contenedores (`docker compose run --rm py31x pytest -q`) y en GitHub Actions.

# 5. Ejercicio 4. Investigación y estado del arte

## 5.1. Investigación

### *5.1.1. El no determinismo*

Un **AFN** es una quíntupla N = (Q, $\Sigma$, $\delta$, $q_0$, F) igual a la de un AFD salvo por la función de transición, que ahora devuelve un conjunto: $\delta$: Q $\times$ $\Sigma$ $\to$ $\mathcal{P}$(Q). Un mismo par puede llevar a varios estados o a ninguno. La extensión a cadenas es $\hat{\delta}(q, \lambda) = \{q\}$ y $\hat{\delta}(q, \omega a) = \bigcup_{p \in \hat{\delta}(q,\omega)} \delta(p, a)$, y N **acepta** $\omega$ si $\hat{\delta}(q_0, \omega) \cap F \neq \varnothing$: basta con que *alguna* de las sucesiones de elecciones termine en un estado final (Hopcroft et al., 2007, sec. 2.3.2). Una manera útil de pensarlo es que el autómata adivina siempre bien, o que explora todas las elecciones en paralelo y gana si alguna gana.

Un **AFN-$\lambda$** permite además transiciones que no consumen símbolo: $\delta$: Q $\times$ ($\Sigma \cup \{\lambda\}$) $\to$ $\mathcal{P}$(Q). Para él, aceptar significa que existe un camino desde $q_0$ hasta un estado de F cuyas etiquetas, al concatenarse, forman $\omega$ (las $\lambda$ no aportan nada a la concatenación) (Sipser, 2013, sec. 1.2).

### *5.1.2. La $\lambda$-clausura*

$\lambda$-cl(q) es el menor conjunto que contiene a q y que es cerrado bajo transiciones $\lambda$: si p $\in$ $\lambda$-cl(q) y r $\in$ $\delta$(p, $\lambda$), entonces r $\in$ $\lambda$-cl(q). Para un conjunto, $\lambda$-cl(S) = $\bigcup_{q \in S}$ $\lambda$-cl(q). Se calcula con un recorrido de grafo restringido a las aristas $\lambda$:

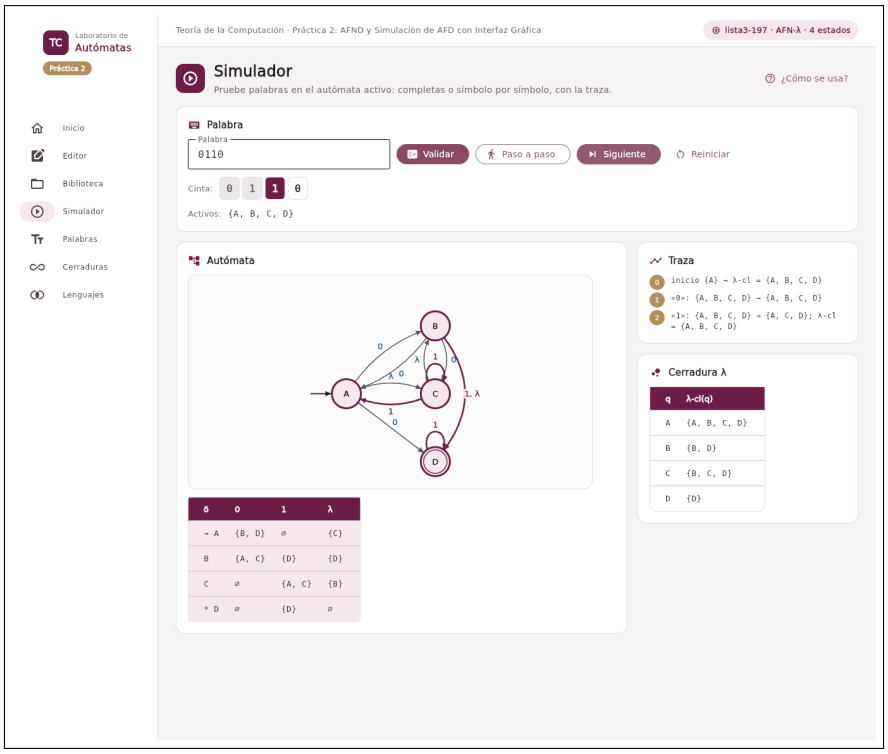



**Figura 10.** Simulador a mitad de una ejecución paso a paso: cinta, activos, transiciones usadas y traza.

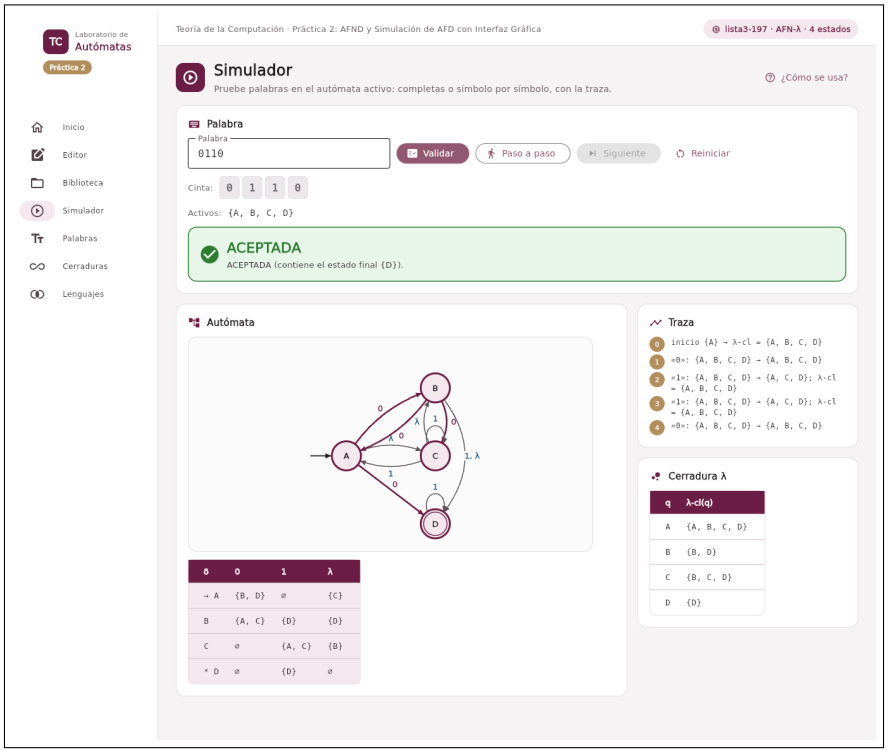



**Figura 11.** Resultado de validar una palabra en un AFN-$\lambda$, con la cerradura $\lambda$ de cada estado.

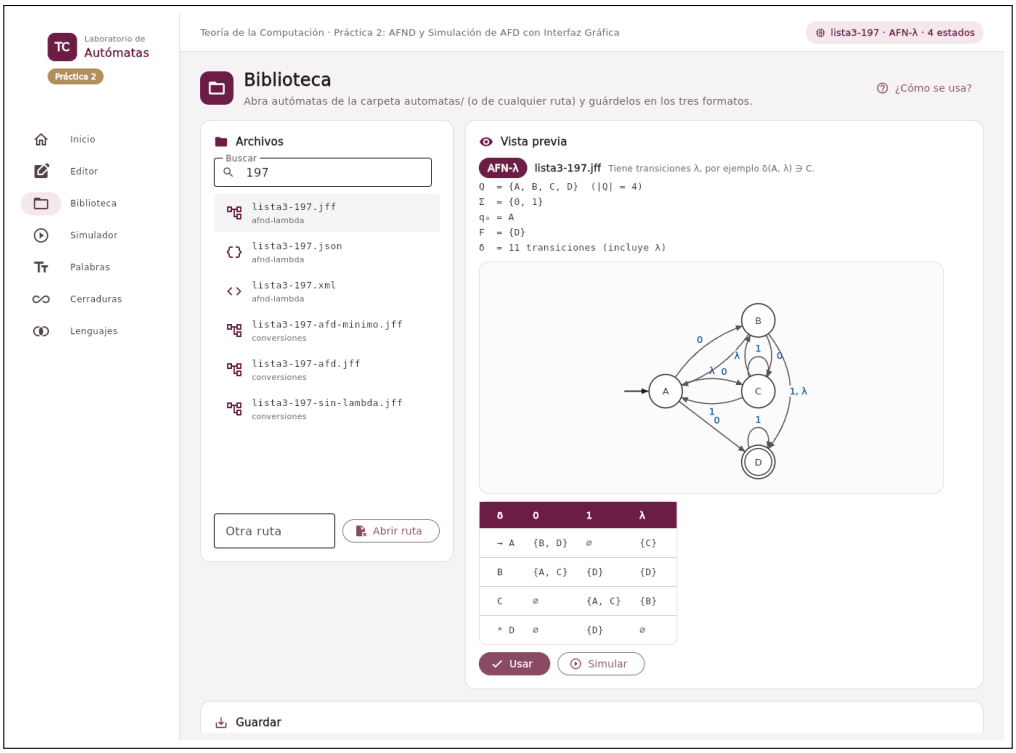



**Figura 12.** Biblioteca: vista previa de un archivo y exportación a los tres formatos.

```
λ-cl(S):
    visitados ← S;  pila ← elementos de S
    mientras pila no esté vacía:
        q ← sacar(pila)
        para cada r en δ(q, λ) que no esté en visitados:
            agregar r a visitados y a pila
    devolver visitados
```

Cada estado entra a la pila una vez y cada arista $\lambda$ se revisa una vez, así que el costo es O(|Q| + |$\delta$_$\lambda$|). Con ella, $\hat{\delta}$(q, $\lambda$) = $\lambda$-cl(q) y $\hat{\delta}$(q, $\omega$a) = $\lambda$-cl($\bigcup$_{p $\in$ $\hat{\delta}$(q, $\omega$)} $\delta$(p, a)) (Hopcroft et al., 2007, sec. 2.5.3).

### *5.1.3. Construcción de subconjuntos y equivalencia AFD–AFN*

**Teorema** (Rabin & Scott, 1959; Hopcroft et al., 2007, sec. 2.3.5). Para todo AFN N existe un AFD D tal que L(D) = L(N).

*Esbozo de la demostración.* Se define D = ($\mathcal{P}$(Q), $\Sigma$, $\delta$_D, {$q_0$}, F_D) con $\delta$_D(S, a) = $\bigcup$_{q$\in$S} $\delta$(q, a) y F_D = {S : S $\cap$ F $\neq$ $\varnothing$}. Por inducción sobre |$\omega$| se prueba que $\hat{\delta}$_D({$q_0$}, $\omega$) = $\hat{\delta}$_N($q_0$,

ω): para $\lambda$ ambos son $\{q_0\}$, y el paso inductivo es la definición de $\delta$_D. Entonces D acepta ω $\Leftrightarrow$ $\delta$_D($\{q_0\}$, ω) $\cap$ F $\neq \varnothing \Leftrightarrow$ N acepta ω. Para un AFN-$\lambda$ basta usar $\lambda$-cl($\{q_0\}$) como inicial y $\lambda$-cl($\bigcup$ $\delta$(q, a)) como transición.

#### *5.1.4. El costo de la conversión*

D puede tener hasta 2^|Q| estados, y la cota es justa: el lenguaje de las cadenas cuyo n-ésimo símbolo desde el final es 1 tiene un AFN de n + 1 estados y ningún AFD de menos de $2^n$ (Hopcroft et al., 2007, sec. 2.3.6). En la práctica se construyen sólo los subconjuntos alcanzables desde el inicial, y suelen ser muchos menos (Ejercicio 2).

#### *5.1.5. Teorema de Kleene*

**Teorema** (Kleene, 1956). Un lenguaje es reconocido por un autómata finito si y sólo si lo describe una expresión regular. En una dirección se construye, por inducción sobre la expresión, un AFN-$\lambda$ para $\varnothing$, $\lambda$ y cada símbolo, y para la unión, la concatenación y la estrella; en la otra, se obtiene una expresión del autómata eliminando estados o con las expresiones $R_{ij}^{(k)}$ (Hopcroft et al., 2007, sec. 3.2). Junto con la equivalencia anterior, AFD, AFN, AFN-$\lambda$ y expresiones regulares describen la misma clase: los lenguajes regulares.

#### *5.1.6. Minimización: Myhill-Nerode y Hopcroft*

Dos cadenas x, y son *indistinguibles* respecto de L (x $\equiv$_L y) si para toda z, xz $\in$ L $\Leftrightarrow$ yz $\in$ L. **Teorema de Myhill-Nerode** (Nerode, 1958): L es regular si y sólo si $\equiv$_L tiene un número finito de clases, y ese número es el de estados del AFD mínimo, que es único salvo el nombre de los estados. El **algoritmo de Hopcroft** (Hopcroft, 1971) encuentra esas clases refinando particiones: empieza con {F, Q − F}, mantiene una lista de bloques «separadores» y, para cada separador B y símbolo a, parte todo bloque cuyos estados van con a unos a B y otros fuera de B. Su truco es agregar a la lista sólo la mitad *más pequeña* de cada bloque partido; así cada estado se procesa O(log |Q|) veces y el algoritmo cuesta O(|Σ|·|Q| log |Q|), frente a O(|Σ|·|Q|²) del refinamiento ingenuo de Moore (Moore, 1971).

#### *5.1.7. Qué significa que el no determinismo no aumente el poder*

Que AFN y AFD reconozcan la misma clase significa que el no determinismo es una herramienta de **descripción**, no de **cómputo**: permite escribir autómatas más pequeños y más fáciles de entender («adivina dónde empieza el patrón»), pero cualquier cosa que haga un AFN puede hacerla un AFD, a costa de quizá exponencialmente más estados. En ingeniería eso se traduce en una elección: se diseña y se razona con el AFN (o con la expresión regular), y se ejecuta con un AFD cuando el tiempo por símbolo importa más que la memoria.

### 5.2. Artículo asignado: Rabin y Scott (1959)

**Dónde se prueba.** En la sección 5, «Nondeterministic operation». El artículo define el autómata no determinista como un sistema (S, M, $S_0$, F) cuya tabla M asigna a cada par (estado, símbolo) un *conjunto* de estados y que admite un *conjunto* de iniciales $S_0$; una cinta se acepta si alguna sucesión de elecciones termina en F. Después construye el autómata determinista cuyos estados son los subconjuntos de S y, en el **teorema 11**, prueba que ambos aceptan las mismas cintas (Rabin & Scott, 1959).

**La construcción en la notación del curso.** Dado N = (Q, $\Sigma$, $\delta$, $q_0$, F), D = ($\mathcal{P}$(Q), $\Sigma$, $\delta$_D, $\{q_0\}$, F_D) con $\delta$_D(S, a) = $\bigcup$_{q∈S} $\delta$(q, a) y F_D = $\{S \subseteq Q : S \cap F \neq \varnothing\}$. Con varios iniciales, como en el artículo, el inicial de D es $S_0$ completo.

**Diferencias de notación.** El artículo llama *tapes* a las cadenas, escribe M en lugar de $\delta$, no incluye el alfabeto en la tupla y admite varios iniciales; el libro de texto usa la quíntupla con $\Sigma$ explícito y un solo $q_0$. La notación del libro resulta más clara: el mismo símbolo $\delta$ sirve para todos los modelos del curso y la extensión $\hat{\delta}$ se define aparte, mientras que el artículo, escrito para lectores de lógica, define la aceptación con sucesiones de estados.

**El Premio Turing.** La mención oficial de la ACM otorga el premio de 1976 a Rabin y Scott por este artículo y destaca que introdujo la idea de las máquinas no deterministas, un concepto que resultó de enorme valor y que siguió inspirando trabajos posteriores (Association for Computing Machinery, 2026). El reconocimiento no se debe a que los AFN reconozcan más lenguajes —el pro-

pio artículo prueba que no—, sino al concepto: el no determinismo es hoy central en la complejidad (la clase NP), en la verificación y en el procesamiento de expresiones regulares.

## 5.3. Dos artículos localizados

### *5.3.1. Saarikivi, Veanes, Wan y Xu (2019)*

1. **Cita.** Saarikivi, O., Veanes, M., Wan, T., y Xu, E. (2019). Symbolic regex matcher. En T. Vojnar y L. Zhang (Eds.), *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2019)* (Lecture Notes in Computer Science, Vol. 11427, pp. 372–378). Springer. https://doi.org/10.1007/978-3-030-17462-0_24
2. **Problema.** El motor de expresiones regulares de .NET usa retroceso (*backtracking*) y algunos patrones tardan un tiempo exponencial; una herramienta de Microsoft que revisa código en la nube en busca de credenciales necesitaba buscar con tiempo lineal.
3. **Método.** SRM construye el autómata sobre la marcha con *derivadas simbólicas de Brzozowski*: el estado es una expresión regular y leer un carácter la transforma en otra. Las clases de caracteres se representan como predicados sobre *minterms* (una partición del alfabeto en clases que se comportan igual) y los estados de AFD ya calculados se guardan en caché.
4. **Resultado.** En dos colecciones de prueba, SRM y RE2 de Google fueron en general más rápidos que el motor de .NET; el tiempo medio de SRM fue comparable al de RE2 con menos casos extremos, aunque su compilación es más lenta. Se adoptó internamente y eliminó el retroceso catastrófico en archivos con líneas largas.
5. **Relación con el temario.** Construcción de subconjuntos perezosa y equivalencia entre expresiones regulares y autómatas.
6. **Aportación.** Es una implementación medida, como pide el enunciado, y muestra la misma decisión que el Ejercicio 3: no construir de antemano el AFD completo, sino sólo los estados que la entrada visita.

#### *5.3.2. Moseley et al. (2023)*

1. **Cita.** Moseley, D., Nishio, M., Perez Rodriguez, J., Saarikivi, O., Toub, S., Veanes, M., Wan, T., y Xu, E. (2023). Derivative based nonbacktracking real-world regex matching with backtracking semantics. *Proceedings of the ACM on Programming Languages*, *7*(PLDI), Artículo 148. https://doi.org/10.1145/3591262
2. **Problema.** Los motores sin retroceso eran inmunes a los ataques de denegación de servicio por expresiones regulares, pero no podían reemplazar a los de retroceso porque no soportaban anclas, contadores ni el mismo resultado de coincidencia.
3. **Método.** Extienden las derivadas de Brzozowski a *derivadas por posición* que manejan anclas, simulan la prioridad de las alternativas del retroceso sin retroceder y buscan en dos pasadas (una hacia adelante para el final de la coincidencia y otra sobre la expresión invertida para el inicio). Demuestran la corrección formalmente e implementan el resultado como la opción `RegexOptions.NonBacktracking` de .NET 7.
4. **Resultado.** En un conjunto industrial de 184 patrones obtienen una aceleración media geométrica de 4.7× sobre el motor compilado de .NET, y en la búsqueda de credenciales sobre 15 GB de código, mejoras de 9 % a 26 %; el beneficio principal es eliminar los casos extremos.
5. **Relación con el temario.** Autómatas finitos deterministas construidos desde expresiones regulares; reflexión (la expresión invertida reconoce $L^R$).
6. **Aportación.** Prueba que la teoría del curso llega a un producto de uso masivo y explica por qué un validador debe reconocer con un autómata y no con un motor de retroceso, idea que se retoma en la Práctica 4.

## 6. Ejercicio 5. Del artículo al autómata

### 6.1. La aplicación elegida

Se eligió el comportamiento de los fantasmas de Pac-Man de Gribkoff (2013, sec. 4, fig. 3). El documento lo describe como un AFD de cuatro estados —*Wander the Maze*, *Chase Pac-Man*,

*Flee Pac-Man* y *Return to Base*— cuyas transiciones dependen de lo que pasa en el juego: ver o perder a Pac-Man, que Pac-Man coma una píldora de poder, que el poder expire, que el fantasma sea comido y que sus ojos lleguen a la base. Bourg y Seemann (2004, cap. 9) usa el mismo ejemplo para introducir las máquinas de estados finitos en videojuegos. El enunciado pide ampliarlo con al menos dos estados y dos eventos.

**Ampliación.** El modelo original omite dos fases que cualquier jugador observa: el fantasma *empieza dentro de la casa central* y sale cuando un temporizador lo libera, y cuando el poder está por terminar el fantasma asustado *parpadea* como advertencia antes de volver a ser peligroso. Se agregan los estados **Casa** y **Titilar** y los eventos **l** (liberación) y **a** (aviso). Con ello cambian dos transiciones del original: los ojos ya no vuelven directamente a vagar, sino a la casa, de donde salen al ser liberados; y el miedo ya no termina de golpe, sino pasando por Titilar.

### 6.2. Definición formal

M = (Q, $\Sigma$, $\delta$, Casa, F) con:

**Tabla 8.** Estados del modelo.

| Estado | Qué representa |
|---|---|
| **Casa** (inicial) | Espera dentro de la casa central; no persigue ni puede ser comido. *Nuevo.* |
| **Vagar** | Recorre el laberinto sin haber visto a Pac-Man (*Wander the Maze*). |
| **Perseguir** | Ve a Pac-Man y lo sigue (*Chase Pac-Man*). |
| **Huir** | Está asustado: Pac-Man comió una píldora de poder (*Flee Pac-Man*). |
| **Titilar** | Sigue asustado, pero el poder está por terminar; parpadea. *Nuevo.* |
| **Regresar** | Pac-Man se lo comió; sólo los ojos vuelven a la casa (*Return to Base*). |

**Alfabeto.** Antes de dibujar hay que decidir qué es un símbolo. Aquí cada símbolo es un **evento del juego** que el fantasma percibe, no una tecla ni un instante de tiempo; una cadena es la bitácora de eventos de una partida, en orden.

**Tabla 9.** Alfabeto $\Sigma$ = {l, v, p, e, a, x, c, b}.

| Símbolo | Evento |
|---|---|
| l | El temporizador libera al fantasma de la casa. *Nuevo.* |
| v | El fantasma ve a Pac-Man. |
| p | El fantasma pierde de vista a Pac-Man. |
| e | Pac-Man come una píldora de poder. |
| a | Aviso: al poder le quedan pocos segundos. *Nuevo.* |
| x | El poder expira. |
| c | Pac-Man se come al fantasma. |
| b | Los ojos llegan a la casa. |

**Función de transición.** Un evento que no altera el comportamiento en un estado deja al fantasma donde está (por ejemplo, ver a Pac-Man mientras huye). Un evento que **no puede ocurrir** en un estado no tiene transición: no se puede perder de vista a quien no se ha visto (p en Vagar), comer a un fantasma que no está asustado (c en Vagar o Perseguir) ni que el poder expire sin el aviso previo (x en Huir).

**Tabla 10.** Función de transición (— = el evento no puede ocurrir en ese estado).

| $\delta$ | l | v | p | e | a | x | c | b |
|---|---|---|---|---|---|---|---|---|
| → Casa | Vagar | — | — | Casa | Casa | Casa | — | — |
| * Vagar | — | Perseguir | — | Huir | Vagar | Vagar | — | — |
| * Perseguir | — | Perseguir | Vagar | Huir | Perseguir | Perseguir | — | — |
| Huir | — | Huir | Huir | Huir | Titilar | — | Regresar | — |
| Titilar | — | Titilar | Titilar | Huir | — | Vagar | Regresar | — |
| Regresar | — | Regresar | Regresar | Regresar | Regresar | Regresar | — | Casa |

**Aceptación.** F = {Vagar, Perseguir}: se aceptan las bitácoras que dejan al fantasma **peligroso para Pac-Man**. Es un AFD con $\delta$ parcial; una bitácora imposible deja vacío el conjunto de estados activos y se rechaza.

El modelo está en `automatas/aplicacion/fantasma-pacman.jff` (y `.json`, `.xml`); se cargó en la Biblioteca y se validó en el Simulador.

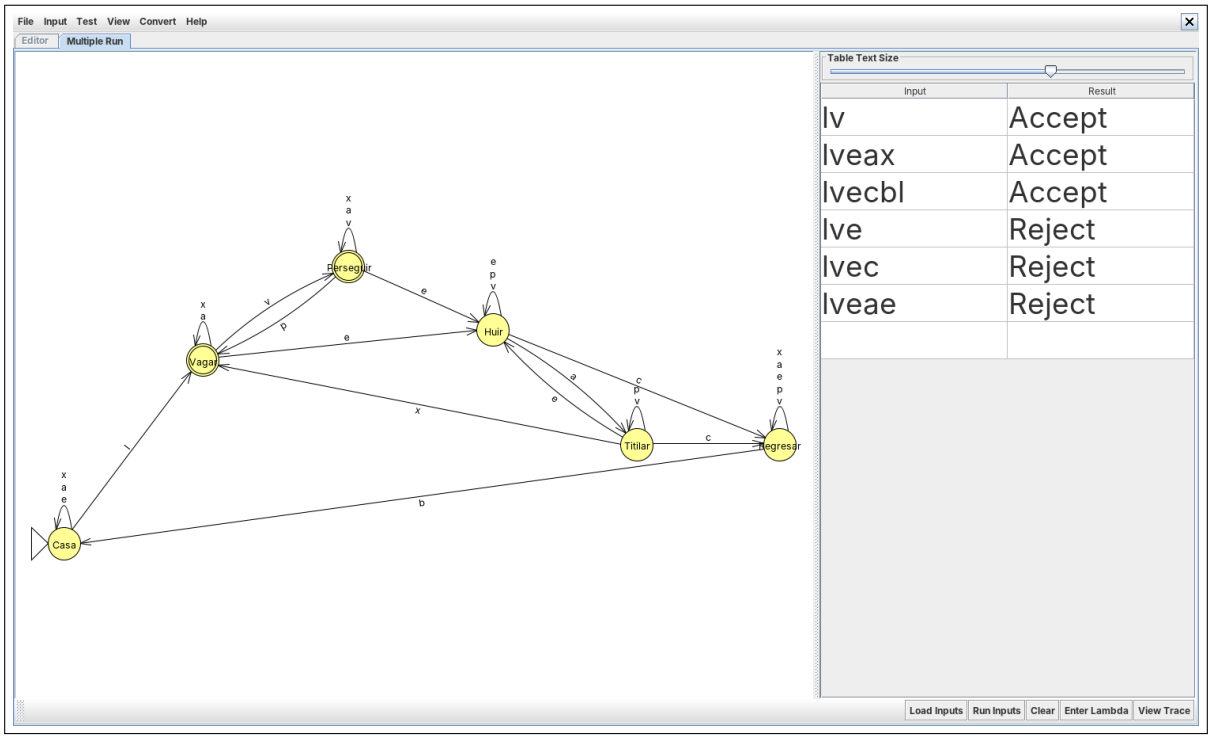



**Figura 13.** El autómata del fantasma en JFLAP.

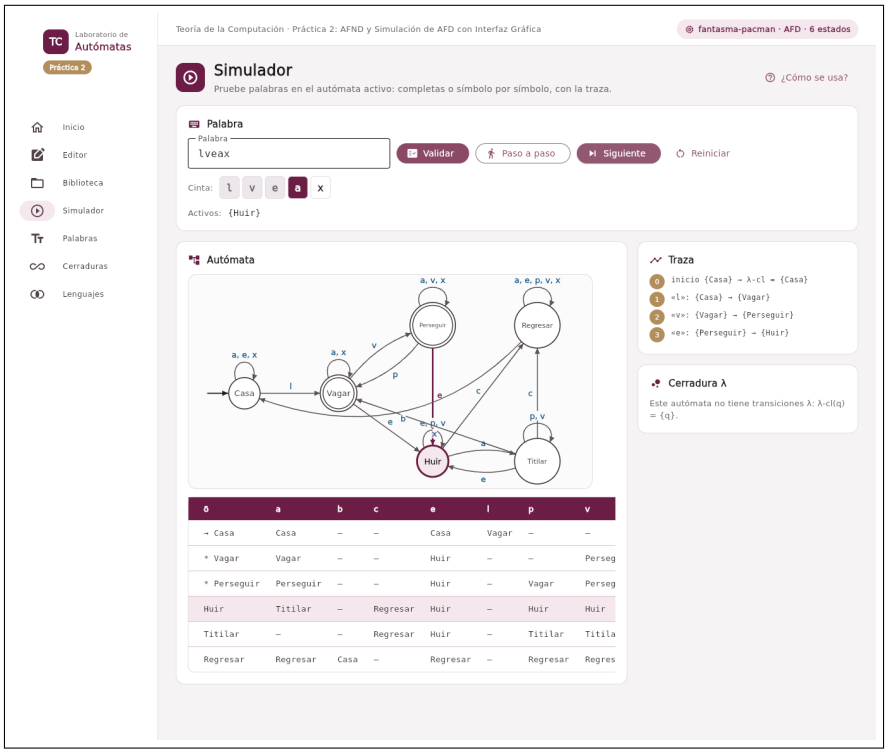



**Figura 14.** El mismo autómata en el simulador propio, validando `lveax` paso a paso.

### 6.3. Cadenas de prueba

**Tabla 11.** Quince cadenas de prueba (`automatas/aplicacion/pruebas.txt`, verificadas en `tests/test_aplicacion.py`).

| Bitácora | Significado | Esperado | Obtenido | Estado final |
|---|---|---|---|---|
| $\lambda$ | Nada ha ocurrido: el fantasma sigue en la casa | rechaza | rechaza | Casa |
| `l` | Sale de la casa | acepta | acepta | Vagar |
| `lv` | Ve a Pac-Man | acepta | acepta | Perseguir |
| `lvp` | Lo pierde | acepta | acepta | Vagar |
| `lve` | Pac-Man come una píldora | rechaza | rechaza | Huir |
| `lveax` | Aviso y fin del poder | acepta | acepta | Vagar |
| `lvec` | Pac-Man se lo come | rechaza | rechaza | Regresar |
| `lvecb` | Los ojos llegan a la casa | rechaza | rechaza | Casa |
| `lvecbl` | Se regenera y sale de nuevo | acepta | acepta | Vagar |
| `lveae` | Otra píldora mientras titila | rechaza | rechaza | Huir |
| `lveaeax` | La segunda píldora también expira | acepta | acepta | Vagar |
| `lp` | Perder a quien no ha visto | rechaza (imposible) | rechaza | $\varnothing$ |
| `lc` | Comer a un fantasma no asustado | rechaza (imposible) | rechaza | $\varnothing$ |
| `lvex` | El poder expira sin aviso | rechaza (imposible) | rechaza | $\varnothing$ |
| `v` | Ver a Pac-Man desde la casa | rechaza (imposible) | rechaza | $\varnothing$ |

### 6.4. La abstracción

El modelo captura con exactitud **el orden en que pueden ocurrir los cambios de comportamiento** del fantasma y, con ello, una propiedad útil para el diseño del juego: después de cualquier bitácora, sabemos si el fantasma puede dañar a Pac-Man, y sabemos qué secuencias de eventos son incoherentes (un fantasma comido sin haber estado asustado delataría un error en el motor del juego). También muestra por qué el original necesitaba ampliarse: con cuatro estados, un fantasma recién comido volvía a vagar sin pasar por la casa, algo que no ocurre en el juego.

Deja fuera casi todo lo que hace interesante al juego. No hay **posiciones** ni laberinto: «ver a

Pac-Man» es un evento atómico, cuando en el juego depende de la geometría. No hay **tiempo**: el aviso y la expiración son eventos, no cuentas regresivas, y el modelo no puede expresar «el poder dura seis segundos» sin un estado por cada tic. No distingue entre los **cuatro fantasmas**, que en el juego tienen estrategias de persecución distintas, ni modela la **dispersión** periódica en la que los fantasmas se retiran a las esquinas. Tampoco hay **salidas**: un AFD sólo acepta o rechaza, de modo que la acción del fantasma (hacia dónde moverse) tendría que modelarse con una máquina de Mealy o de Moore. Esos límites no son defectos del diseño sino del modelo: un autómata finito tiene memoria acotada y sin salida, y cualquier cosa que dependa de cantidades no acotadas o de producir respuestas exige otro modelo.

## 7. Propuesta de trabajo

**Título tentativo.** Validación de datos de formularios con autómatas mínimos: tiempo lineal garantizado y la misma semántica en el cliente y en el servidor.

**Problema.** Los sistemas web validan los datos (correos, CURP, teléfonos, fechas) con expresiones regulares escritas una vez y copiadas entre lenguajes: JavaScript en el navegador y Python o Java en el servidor. Davis et al. (2019) mostraron que cerca del 15 % de las expresiones cambia de significado al pasar de un lenguaje a otro y que muchas tardan un tiempo superlineal en algún motor; un validador que acepta un dato en el navegador y lo rechaza en el servidor, o que se congela con una entrada maliciosa, es un error real. Los formatos de esos campos son, casi siempre, **lenguajes regulares**, así que admiten un modelo formal único: un AFD.

**Clase de máquina.** Con lo visto hasta ahora basta el autómata finito: se escribe la expresión en la notación del curso (sin referencias hacia atrás), se convierte en AFN-$\lambda$, se determiniza y se minimiza, y el AFD mínimo se usa como especificación ejecutable en ambos lados. Prevé requerir **autómatas de pila** para los datos con anidamiento, como un objeto JSON o una expresión aritmética en un campo de fórmula, que no son regulares; no prevé máquinas de Turing.

**Sustento.**

- Davis et al. (2019) cuantifican el problema de portabilidad y de rendimiento de las expresiones

regulares.

- Moseley et al. (2023) muestran que un motor basado en autómatas puede reemplazar a uno de retroceso en un producto de uso masivo sin perder rendimiento.

**Qué se construiría.** Un generador que reciba un catálogo de formatos en la notación del curso y produzca, para cada uno, el AFD mínimo y su código en JavaScript y en Python (una tabla de transiciones y un ciclo de lectura), más un informe con el número de estados.

**Cómo se comprobaría.** (1) Equivalencia: para cada formato, comparar el AFD con `re.fullmatch` y con el motor de JavaScript sobre miles de cadenas generadas y mutadas, como en las pruebas de esta práctica; cualquier diferencia indica un error de traducción. (2) Tiempo: medir el tiempo de validación contra la longitud de entradas diseñadas para provocar retroceso y comprobar que crece linealmente. (3) Uso: integrarlo en un formulario de prueba y verificar que cliente y servidor dan el mismo veredicto.

## 8. Conclusiones

Trabajar con una rama por funcionalidad y con integración continua en tres versiones de Python hizo que cada parte del simulador entrara a `main` ya probada; reutilizar `lenguajes.py` sin tocarlo confirmó que la separación entre núcleo e interfaz de la Práctica 1 era la correcta.

Clasificar los 140 autómatas de las listas por su función de transición mostró un patrón: el no determinismo aparece cuando el enunciado obliga a adivinar una posición o a elegir entre condiciones, y las transiciones $\lambda$ cuando el lenguaje se arma con uniones, concatenaciones y cerraduras. En las 48 conversiones el AFD nunca pasó del 62.5 % de la cota $2^n$, y en el caso más grande se quedó en 32 de más de 67 millones de subconjuntos posibles; la cota es real, pero sólo la alcanzan familias construidas para ello.

En el simulador, la jerarquía `Automata` $\rightarrow$ `AFNLambda` $\rightarrow$ `AFN` $\rightarrow$ `AFD` hizo explícito que cada clase restringe a la anterior y que el tipo se deduce de $\delta$. Seguir el conjunto de estados activos permitió mostrar las bifurcaciones y la cerradura $\lambda$ en cada paso sin pagar el costo exponencial de convertir, y las pruebas de ida y vuelta garantizan que los tres formatos guardan la misma quíntupla.

La lectura de Rabin y Scott y de los dos trabajos de Microsoft mostró la misma idea en dos épocas: el no determinismo no reconoce más lenguajes, pero cambia el tamaño de las descripciones, y los motores modernos construyen sólo los estados que la entrada visita. El modelo del fantasma de Pac-Man dejó claro el alcance de la abstracción: un autómata finito captura con precisión el orden válido de los eventos, pero no el tiempo, el espacio ni las acciones del sistema real.

## Referencias

Association for Computing Machinery. (2026). *Michael O. Rabin. A.M. Turing Award laureate*. Consultado el 6 de noviembre de 2026, desde https://amturing.acm.org/award_recipient/rabin_9681074

Bourg, D. M., & Seemann, G. (2004). *AI for game developers*. O'Reilly Media.

Davis, J. C., Michael IV, L. G., Coghlan, C. A., Servant, F., & Lee, D. (2019). Why aren't regular expressions a lingua franca? An empirical study on the re-use and portability of regular expressions. *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 443-454. https://doi.org/10.1145/3338906.3338909

GitHub Inc. (2026). *GitHub Actions documentation*. Consultado el 6 de noviembre de 2026, desde https://docs.github.com/en/actions

Gribkoff, E. (2013). *Applications of deterministic finite automata* [Documento de curso, ECS 120]. https://www.cs.ucdavis.edu/~rogaway/classes/120/spring13/eric-dfa.pdf

Hopcroft, J. E. (1971). An n log n algorithm for minimizing states in a finite automaton. En Z. Kohavi & A. Paz (Eds.), *Theory of Machines and Computations* (pp. 189-196). Academic Press. https://doi.org/10.1016/B978-0-12-417750-5.50022-1

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3.ª ed.). Pearson Addison-Wesley.

Kleene, S. C. (1956). Representation of events in nerve nets and finite automata. En C. E. Shannon & J. McCarthy (Eds.), *Automata Studies* (pp. 3-41). Princeton University Press. https://doi.org/10.1515/9781400882618-002

Moore, F. R. (1971). On the bounds for state-set size in the proofs of equivalence between deterministic, nondeterministic, and two-way finite automata. *IEEE Transactions on Computers*, *C-20*(10), 1211-1214. https://doi.org/10.1109/T-C.1971.223108

Moseley, D., Nishio, M., Perez Rodriguez, J., Saarikivi, O., Toub, S., Veanes, M., Wan, T., & Xu, E. (2023). Derivative based nonbacktracking real-world regex matching with backtracking

semantics. *Proceedings of the ACM on Programming Languages*, *7*(PLDI), Artículo 148. https://doi.org/10.1145/3591262

Nerode, A. (1958). Linear automaton transformations. *Proceedings of the American Mathematical Society*, *9*(4), 541-544. https://doi.org/10.1090/S0002-9939-1958-0135681-9

Rabin, M. O., & Scott, D. (1959). Finite automata and their decision problems. *IBM Journal of Research and Development*, *3*(2), 114-125. https://doi.org/10.1147/rd.32.0114

Sipser, M. (2013). *Introduction to the theory of computation* (3.ª ed.). Cengage Learning.

# A. Anexo A. Clasificación de los ejercicios

**Tabla 12.** Clasificación de los ejercicios de las Listas 2 y 3.

| Ejercicio | Sección | Justificación |
|---|---|---|
| Lista 2 · Ejercicio 094 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 095 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 096 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 097 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 098 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 099 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 100 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 101 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 102 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 103 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 106 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 104 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 105 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 107 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 109 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 108 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 110 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 111 | AFD | Se decide con información acotada del prefijo: basta un AFD. |

| Ejercicio | Sección | Justificación |
|---|---|---|
| Lista 2 · Ejercicio 112 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 113 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 114 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 115 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 116 | — | No regular: exige comparar el número de aes con el de ces (lema de bombeo). |
| Lista 2 · Ejercicio 117 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 118 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 119 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 120 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 121 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 122 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 123 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 124 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 125 | AFD | Propiedad de los AFD completos; se ilustra con un AFD. |
| Lista 2 · Ejercicio 126 | AFD | Propiedad de los AFD; se ilustra con un AFD. |
| Lista 2 · Ejercicio 127 | AFD | Propiedad de los AFD; se ilustra con un AFD. |
| Lista 2 · Ejercicio 128 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 129 | AFD | El enunciado da la quíntupla de un AFD. |
| Lista 2 · Ejercicio 130 | AFD | El enunciado da la quíntupla de un AFD. |
| Lista 2 · Ejercicio 131 | AFD | El enunciado da la quíntupla de un AFD. |

| Ejercicio | Sección | Justificación |
|---|---|---|
| Lista 2 · Ejercicio 132 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 133 | AFD | El enunciado da la quíntupla de un AFD. |
| Lista 2 · Ejercicio 134 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 135 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 136 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 137 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 138 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 139 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 140a | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 140b | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 141 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 142a | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 142b | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 143 | — | Sin figuras en el enunciado. |
| Lista 2 · Ejercicio 144a | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 144b | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 144c | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 144d | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 144e | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 145 | AFD | Se decide con información acotada del prefijo: basta un AFD. |

| Ejercicio | Sección | Justificación |
|---|---|---|
| Lista 2 · Ejercicio 146 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 147 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 148 | AFD | Se decide con información acotada del prefijo: basta un AFD. |
| Lista 2 · Ejercicio 149 | AFD | El enunciado da la tabla de un AFD. |
| Lista 2 · Ejercicio 150 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 151 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 152 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 153 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 154 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 155 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 156 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 157 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 158 | AFD | Función de transición determinista (parcial), aunque aparezca en la sección de AFN. |
| Lista 3 · Ejercicio 159 | AFD | Función de transición determinista (parcial), aunque aparezca en la sección de AFN. |
| Lista 3 · Ejercicio 160 | AFD | Función de transición determinista (parcial), aunque aparezca en la sección de AFN. |
| Lista 3 · Ejercicio 161 | AFD | Función de transición determinista (parcial), aunque aparezca en la sección de AFN. |
| Lista 3 · AFN del problema 094 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 095 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 096 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 097 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |

| Ejercicio | Sección | Justificación |
|---|---|---|
| Lista 3 · AFN del problema 098 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 099 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 100 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 101 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 102 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 103 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 104 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 105 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 106 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 107 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 108 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 109 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 110 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 111 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 112 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 113 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 114 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 115 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 116 | — | No regular. |
| Lista 3 · AFN del problema 117 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |

| Ejercicio | Sección | Justificación |
|---|---|---|
| Lista 3 · AFN del problema 118 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 119 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 120 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · AFN del problema 121 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 122 | AFD | Pide un AFN; sin una versión no determinista más simple, se usa el AFD parcial. |
| Lista 3 · AFN del problema 123 | AFND | Adivina una posición o una rama: AFN. |
| Lista 3 · Ejercicio 189 | AFND | Unión de dos búsquedas: AFN. |
| Lista 3 · Ejercicio 190 | AFND | Intersección de dos AFN. |
| Lista 3 · Ejercicio 191 | — | Encabezado de los ejercicios 192-195. |
| Lista 3 · Ejercicio 192 | AFND | Hay pares (estado, símbolo) con varios destinos. |
| Lista 3 · Ejercicio 193 | AFND | Hay pares (estado, símbolo) con varios destinos. |
| Lista 3 · Ejercicio 194 | AFND | Hay pares (estado, símbolo) con varios destinos. |
| Lista 3 · Ejercicio 195 | AFND | Hay pares (estado, símbolo) con varios destinos. |
| Lista 3 · Ejercicio 196 | AFND-$\lambda$ | La tabla tiene transiciones $\lambda$. |
| Lista 3 · Ejercicio 197 | AFND-$\lambda$ | La tabla tiene transiciones $\lambda$. |
| Lista 3 · Ejercicio 198 | AFND-$\lambda$ | La tabla tiene transiciones $\lambda$. |
| Lista 3 · Ejercicio 199 | AFND-$\lambda$ | La tabla tiene transiciones $\lambda$. |
| Lista 3 · Ejercicio 200 | AFND-$\lambda$ | La tabla tiene transiciones $\lambda$. |
| Lista 3 · Ejercicio 201-235 | — | Pertenencia a expresiones regulares (AFN-$\lambda$ de Thompson). |
| Lista 3 · Ejercicio 236 | AFND-$\lambda$ | Se parte de una expresión regular: su autómata natural es el AFN-$\lambda$ de Thompson. |

| Ejercicio | Sección | Justificación |
|---|---|---|
| Lista 3 · Ejercicio 237 | AFND-$\lambda$ | Se parte de una expresión regular: su autómata natural es el AFN-$\lambda$ de Thompson. |
| Lista 3 · Ejercicio 238 | AFND-$\lambda$ | Se parte de una expresión regular: su autómata natural es el AFN-$\lambda$ de Thompson. |
| Lista 3 · Ejercicio 239 | AFND-$\lambda$ | Se parte de una expresión regular: su autómata natural es el AFN-$\lambda$ de Thompson. |
| Lista 3 · Ejercicio 240 | AFND-$\lambda$ | Se parte de una expresión regular: su autómata natural es el AFN-$\lambda$ de Thompson. |
| Lista 3 · Ejercicio 241 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 242 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 243 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 244 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 245 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 246 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 247 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 248 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 249 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 250 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 251 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 252 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 253 | AFND-$\lambda$ | Expresión regular $\to$ AFN-$\lambda$. |
| Lista 3 · Ejercicio 254 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 255 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 256 | AFD | El enunciado da la tabla de un AFD. |

| Ejercicio | Sección | Justificación |
|---|---|---|
| Lista 3 · Ejercicio 257 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 258 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 259 | AFD | El enunciado da la tabla de un AFD. |
| Lista 3 · Ejercicio 260 | AFND | AFN con varios destinos por par (estado, símbolo). |
| Lista 3 · Ejercicio 261 | AFND | AFN con varios destinos por par (estado, símbolo). |
| Lista 3 · Ejercicio 262 | AFD | Diseño natural con dos ramas (AFN). |
| Lista 3 · Ejercicio 263 | AFND | AFN con varios destinos por par (estado, símbolo). |

## B. Anexo B. Cerraduras $\lambda$

Para cada AFN-$\lambda$ se indica por qué hacen falta las transiciones $\lambda$ y la cerradura de cada estado. La cerradura es lo primero que calcula el simulador: el inicio de la lectura no es $\{q_0\}$ sino $\lambda$-cl($q_0$), y después de cada símbolo se vuelve a cerrar el conjunto de estados activos.

**Lista 3 · Ejercicio 196. Cerraduras:** Cl(A) = {A, B}, Cl(B) = {B}, Cl(C) = {C}. **a)** $\delta$(A, 110) = $\varnothing$. **b)** Cl({A}) = {A, B, C}. **c)** Cl({B}) = {B, C}. **d)** Cl({A, B}) = {A, B, C}. **e)** $\delta$(A, 0) = {A} (sin cerradura). **AFD equivalente:** 3 estados.

$\lambda$-cl(A) = {A, B} · $\lambda$-cl(B) = {B} · $\lambda$-cl(C) = {C}

**Lista 3 · Ejercicio 197. Cerraduras:** Cl(A) = {A, B, C, D}, Cl(B) = {B, D}, Cl(C) = {B, C, D}, Cl(D) = {D}. **a)** Cl({A}) = {A, B, C, D}. **b)** Cl({B, D}) = {B, C, D}. **c)** Cl({A, B, C}) = {A, B, C, D}. **d)** $\delta$(A, 0) = {B, D} (sin cerradura). **e)** $\delta$({A}, 010) = {A, B, C, D}. **AFD equivalente:** 1 estados.

$\lambda$-cl(A) = {A, B, C, D} · $\lambda$-cl(B) = {B, D} · $\lambda$-cl(C) = {B, C, D} · $\lambda$-cl(D) = {D}

**Lista 3 · Ejercicio 198. Cerraduras:** Cl(A) = {A, B, C}, Cl(B) = {B}, Cl(C) = {B, C}, Cl(D) = {D}. **a)** Cl({A}) = {A, B, C}. **b)** Cl({C, D}) = {B, C, D}. **c)** Cl({A, C, D}) = {A, B, C, D}. **d)** $\delta$(B, 0111) = {B, C, D}. **AFD equivalente:** 3 estados.

$\lambda$-cl(A) = {A, B, C} · $\lambda$-cl(B) = {B} · $\lambda$-cl(C) = {B, C} · $\lambda$-cl(D) = {D}

**Lista 3 · Ejercicio 199. Cerraduras:** Cl(A) = {A, B, C, E}, Cl(B) = {B}, Cl(C) = {C, E}, Cl(D) = {D}, Cl(E) = {E}. **a)** Cl({A}) = {A, B, C, E}. **b)** Cl({C}) = {C, E}. **c)** $\delta$(A, 011) = {D}. **AFD equivalente:** 5 estados.

$\lambda$-cl(A) = {A, B, C, E} · $\lambda$-cl(B) = {B} · $\lambda$-cl(C) = {C, E} · $\lambda$-cl(D) = {D} · $\lambda$-cl(E) = {E}

**Lista 3 · Ejercicio 200. Cerraduras:** Cl(A) = {A, B}, Cl(B) = {B}, Cl(C) = {C, E}, Cl(D) = {D, E, F}, Cl(E) = {E}, Cl(F) = {E, F}. **a)** Cl({A}) = {A, B, C, E}. **b)** Cl({C, D}) = {C, D, E, F}. **c)** $\delta$(C, 1011) = {A, B, C, D, E, F}. **AFD equivalente:** 6 estados.

$\lambda$-cl(A) = {A, B} · $\lambda$-cl(B) = {B} · $\lambda$-cl(C) = {C, E} · $\lambda$-cl(D) = {D, E, F} · $\lambda$-cl(E) = {E} · $\lambda$-cl(F) = {E, F}

**Lista 3 · Ejercicio 236. Expresión regular:** `(0 + 1)*001(0 + 1)*`.

$\lambda$-cl(q6) = {q6, q4, q0, q2, q7, q8} · $\lambda$-cl(q4) = {q4, q0, q2} · $\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q4, q0, q1, q2, q5, q7, q8} · $\lambda$-cl(q2) = {q2} · $\lambda$-cl(q3) = {q4, q0, q2, q3, q5, q7, q8} · $\lambda$-cl(q5) = {q4, q0, q2, q5, q7, q8}

· λ-cl(q7) = {q7, q8} · λ-cl(q8) = {q8} · λ-cl(q9) = {q9, q10} · λ-cl(q10) = {q10} · λ-cl(q11) = {q11, q12} · λ-cl(q12) = {q12} · λ-cl(q13) = {q13, q20, q18, q14, q16, q21} · λ-cl(q20) = {q20, q18, q14, q16, q21} · λ-cl(q18) = {q18, q14, q16} · λ-cl(q14) = {q14} · λ-cl(q15) = {q18, q14, q15, q16, q19, q21} · λ-cl(q16) = {q16} · λ-cl(q17) = {q18, q14, q16, q17, q19, q21} · λ-cl(q19) = {q18, q14, q16, q19, q21} · λ-cl(q21) = {q21}

**Lista 3 · Ejercicio 237. Expresión regular:** `(0 + 1)*010`.

λ-cl(q6) = {q6, q4, q0, q2, q7, q8} · λ-cl(q4) = {q4, q0, q2} · λ-cl(q0) = {q0} · λ-cl(q1) = {q4, q0, q1, q2, q5, q7, q8} · λ-cl(q2) = {q2} · λ-cl(q3) = {q4, q0, q2, q3, q5, q7, q8} · λ-cl(q5) = {q4, q0, q2, q5, q7, q8} · λ-cl(q7) = {q7, q8} · λ-cl(q8) = {q8} · λ-cl(q9) = {q9, q10} · λ-cl(q10) = {q10} · λ-cl(q11) = {q11, q12} · λ-cl(q12) = {q12} · λ-cl(q13) = {q13}

**Lista 3 · Ejercicio 238. Expresión regular:** `(0 + 1)(0 + 1)(0 + 1)(λ + (0 + 1)(0 + 1))`.

λ-cl(q4) = {q4, q0, q2} · λ-cl(q0) = {q0} · λ-cl(q1) = {q1, q5, q10, q6, q8} · λ-cl(q2) = {q2} · λ-cl(q3) = {q3, q5, q10, q6, q8} · λ-cl(q5) = {q5, q10, q6, q8} · λ-cl(q10) = {q10, q6, q8} · λ-cl(q6) = {q6} · λ-cl(q7) = {q7, q11, q16, q12, q14} · λ-cl(q8) = {q8} · λ-cl(q9) = {q9, q11, q16, q12, q14} · λ-cl(q11) = {q11, q16, q12, q14} · λ-cl(q16) = {q16, q12, q14} · λ-cl(q12) = {q12} · λ-cl(q13) = {q13, q17, q32, q18, q19, q24, q20, q22, q33} · λ-cl(q14) = {q14} · λ-cl(q15) = {q15, q17, q32, q18, q19, q24, q20, q22, q33} · λ-cl(q17) = {q17, q32, q18, q19, q24, q20, q22, q33} · λ-cl(q32) = {q32, q18, q19, q24, q20, q22, q33} · λ-cl(q18) = {q18, q19, q33} · λ-cl(q19) = {q19, q33} · λ-cl(q24) = {q24, q20, q22} · λ-cl(q20) = {q20} · λ-cl(q21) = {q21, q25, q30, q26, q28} · λ-cl(q22) = {q22} · λ-cl(q23) = {q23, q25, q30, q26, q28} · λ-cl(q25) = {q25, q30, q26, q28} · λ-cl(q30) = {q30, q26, q28} · λ-cl(q26) = {q26} · λ-cl(q27) = {q27, q31, q33} · λ-cl(q28) = {q28} · λ-cl(q29) = {q29, q31, q33} · λ-cl(q31) = {q31, q33} · λ-cl(q33) = {q33}

**Lista 3 · Ejercicio 239. Expresión regular:** `01(0 + 1)* + (0 + 1)*10`.

λ-cl(q24) = {q24, q0, q18, q16, q12, q14, q19, q20} · λ-cl(q0) = {q0} · λ-cl(q1) = {q1, q2} · λ-cl(q2) = {q2} · λ-cl(q3) = {q3, q10, q8, q4, q6, q11, q25} · λ-cl(q10) = {q10, q8, q4, q6, q11, q25} · λ-cl(q8) = {q8, q4, q6} · λ-cl(q4) = {q4} · λ-cl(q5) = {q8, q4, q5, q6, q9, q11, q25} · λ-cl(q6) = {q6} · λ-cl(q7) = {q8, q4, q6, q7, q9, q11, q25} · λ-cl(q9) = {q8, q4, q6, q9, q11, q25} · λ-cl(q11) = {q11, q25} · λ-cl(q18) = {q18, q16, q12, q14, q19, q20} · λ-cl(q16) = {q16, q12, q14} · λ-cl(q12) = {q12} · λ-cl(q13) = {q16, q12, q13, q14, q17, q19, q20} · λ-cl(q14) = {q14} · λ-cl(q15) = {q16, q12, q14, q15, q17, q19, q20} · λ-cl(q17) = {q16, q12, q14, q17, q19, q20} · λ-cl(q19) = {q19, q20} · λ-cl(q20) = {q20} · λ-cl(q21) = {q21, q22} · λ-cl(q22) = {q22} · λ-cl(q23) = {q23, q25} · λ-cl(q25) = {q25}

**Lista 3 · Ejercicio 240. Expresión regular:** `((0 + 1)(0 + 1))*01((0 + 1)(0 + 1))* + (0 + 1)((0 + 1)(0 + 1))*01((0 + 1)(0 + 1))*(0 + 1)`. Si w = x01y tiene longitud par, |x| + |y| es par, así que |x| y |y| son ambos pares o ambos impares; cada término cubre uno de los dos casos.

λ-cl(q76) = {q76, q12, q4, q0, q2, q13, q14, q36, q32, q34} · λ-cl(q12) = {q12, q4, q0, q2, q13, q14} · λ-cl(q4) = {q4, q0, q2} · λ-cl(q0) = {q0} · λ-cl(q1) = {q1, q5, q10, q6, q8} · λ-cl(q2) = {q2} · λ-cl(q3) = {q3, q5, q10, q6, q8} · λ-cl(q5) = {q5, q10, q6, q8} · λ-cl(q10) = {q10, q6, q8} · λ-cl(q6) = {q6} · λ-cl(q7) = {q4, q0, q2, q7, q11, q13, q14} · λ-cl(q8) = {q8} · λ-cl(q9) = {q4, q0, q2, q9, q11, q13, q14} · λ-cl(q11) = {q4, q0, q2, q11, q13, q14} · λ-cl(q13) = {q13, q14} · λ-cl(q14) = {q14} · λ-cl(q15) = {q15, q16} · λ-cl(q16) = {q16} · λ-cl(q17) = {q17, q30, q22, q18, q20, q31, q77} · λ-cl(q30) = {q30, q22, q18, q20, q31, q77} · λ-cl(q22) = {q22, q18, q20} · λ-cl(q18) = {q18} · λ-cl(q19) = {q19, q23, q28, q24, q26} · λ-cl(q20) = {q20} · λ-cl(q21) = {q21, q23, q28, q24, q26} · λ-cl(q23) = {q23, q28, q24, q26} · λ-cl(q28) = {q28, q24, q26} · λ-cl(q24) = {q24} · λ-cl(q25) = {q22, q18, q20, q25, q29, q31, q77} · λ-cl(q26) = {q26} · λ-cl(q27) = {q22, q18, q20, q27, q29, q31, q77} · λ-cl(q29) = {q22, q18, q20, q29, q31, q77} · λ-cl(q31) = {q31, q77} · λ-cl(q36) = {q36, q32, q34} · λ-cl(q32) = {q32} · λ-cl(q33) = {q33, q37, q50, q42, q38, q40, q51, q52} · λ-cl(q34) = {q34} · λ-cl(q35) = {q35, q37, q50, q42, q38, q40, q51, q52} · λ-cl(q37) = {q37, q50, q42, q38, q40, q51, q52} · λ-cl(q50) = {q50, q42, q38, q40, q51, q52} · λ-cl(q42) = {q42, q38, q40} · λ-cl(q38) = {q38} · λ-cl(q39) = {q39, q43, q48, q44, q46} · λ-cl(q40) = {q40} · λ-cl(q41) = {q41, q43, q48, q44, q46} ·

$\lambda$-cl(q43) = {q43, q48, q44, q46} · $\lambda$-cl(q48) = {q48, q44, q46} · $\lambda$-cl(q44) = {q44} · $\lambda$-cl(q45) = {q42, q38, q40, q45, q49, q51, q52} · $\lambda$-cl(q46) = {q46} · $\lambda$-cl(q47) = {q42, q38, q40, q47, q49, q51, q52} · $\lambda$-cl(q49) = {q42, q38, q40, q49, q51, q52} · $\lambda$-cl(q51) = {q51, q52} · $\lambda$-cl(q52) = {q52} · $\lambda$-cl(q53) = {q53, q54} · $\lambda$-cl(q54) = {q54} · $\lambda$-cl(q55) = {q55, q68, q60, q56, q58, q69, q74, q70, q72} · $\lambda$-cl(q68) = {q68, q60, q56, q58, q69, q74, q70, q72} · $\lambda$-cl(q60) = {q60, q56, q58} · $\lambda$-cl(q56) = {q56} · $\lambda$-cl(q57) = {q57, q61, q66, q62, q64} · $\lambda$-cl(q58) = {q58} · $\lambda$-cl(q59) = {q59, q61, q66, q62, q64} · $\lambda$-cl(q61) = {q61, q66, q62, q64} · $\lambda$-cl(q66) = {q66, q62, q64} · $\lambda$-cl(q62) = {q62} · $\lambda$-cl(q63) = {q60, q56, q58, q63, q67, q69, q74, q70, q72} · $\lambda$-cl(q64) = {q64} · $\lambda$-cl(q65) = {q60, q56, q58, q65, q67, q69, q74, q70, q72} · $\lambda$-cl(q67) = {q60, q56, q58, q67, q69, q74, q70, q72} · $\lambda$-cl(q69) = {q69, q74, q70, q72} · $\lambda$-cl(q74) = {q74, q70, q72} · $\lambda$-cl(q70) = {q70} · $\lambda$-cl(q71) = {q71, q75, q77} · $\lambda$-cl(q72) = {q72} · $\lambda$-cl(q73) = {q73, q75, q77} · $\lambda$-cl(q75) = {q75, q77} · $\lambda$-cl(q77) = {q77}

**Lista 3** · **Ejercicio 241.** AFN-$\lambda$ de 6 estados; AFD mínimo de 3 estados.

$\lambda$-cl(q4) = {q4, q0, q2} · $\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q1, q5} · $\lambda$-cl(q2) = {q2} · $\lambda$-cl(q3) = {q3, q5} · $\lambda$-cl(q5) = {q5}

**Lista 3** · **Ejercicio 242.** AFN-$\lambda$ de 8 estados; AFD mínimo de 4 estados.

$\lambda$-cl(q6) = {q6, q2, q0, q3, q4, q7} · $\lambda$-cl(q2) = {q2, q0, q3, q7} · $\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q0, q1, q3, q7} · $\lambda$-cl(q3) = {q3, q7} · $\lambda$-cl(q4) = {q4} · $\lambda$-cl(q5) = {q5, q7} · $\lambda$-cl(q7) = {q7}

**Lista 3** · **Ejercicio 243.** AFN-$\lambda$ de 8 estados; AFD mínimo de 1 estados.

$\lambda$-cl(q6) = {q6, q4, q0, q2, q7} · $\lambda$-cl(q4) = {q4, q0, q2} · $\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q4, q0, q1, q2, q5, q7} · $\lambda$-cl(q2) = {q2} · $\lambda$-cl(q3) = {q4, q0, q2, q3, q5, q7} · $\lambda$-cl(q5) = {q4, q0, q2, q5, q7} · $\lambda$-cl(q7) = {q7}

**Lista 3** · **Ejercicio 244.** AFN-$\lambda$ de 8 estados; AFD mínimo de 4 estados.

$\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q1, q6, q2, q4} · $\lambda$-cl(q6) = {q6, q2, q4} · $\lambda$-cl(q2) = {q2} · $\lambda$-cl(q3) = {q3, q7} · $\lambda$-cl(q4) = {q4} · $\lambda$-cl(q5) = {q5, q7} · $\lambda$-cl(q7) = {q7}

**Lista 3** · **Ejercicio 245.** AFN-$\lambda$ de 10 estados; AFD mínimo de 3 estados.

$\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q1, q8, q6, q2, q4, q9} · $\lambda$-cl(q8) = {q8, q6, q2, q4, q9} · $\lambda$-cl(q6) = {q6, q2, q4} · $\lambda$-cl(q2) = {q2} · $\lambda$-cl(q3) = {q6, q2, q3, q4, q7, q9} · $\lambda$-cl(q4) = {q4} · $\lambda$-cl(q5) = {q6, q2, q4, q5, q7, q9} · $\lambda$-cl(q7) = {q6, q2, q4, q7, q9} · $\lambda$-cl(q9) = {q9}

**Lista 3** · **Ejercicio 246.** AFN-$\lambda$ de 10 estados; AFD mínimo de 4 estados.

$\lambda$-cl(q2) = {q2, q0, q3, q8, q4, q6} · $\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q0, q1, q3, q8, q4, q6} · $\lambda$-cl(q3) = {q3, q8, q4, q6} · $\lambda$-cl(q8) = {q8, q4, q6} · $\lambda$-cl(q4) = {q4} · $\lambda$-cl(q5) = {q5, q9} · $\lambda$-cl(q6) = {q6} · $\lambda$-cl(q7) = {q7, q9} · $\lambda$-cl(q9) = {q9}

**Lista 3** · **Ejercicio 247.** AFN-$\lambda$ de 12 estados; AFD mínimo de 5 estados.

$\lambda$-cl(q10) = {q10, q4, q0, q5, q8, q6, q9, q11} · $\lambda$-cl(q4) = {q4, q0, q5, q11} · $\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q1, q2} · $\lambda$-cl(q2) = {q2} · $\lambda$-cl(q3) = {q0, q3, q5, q11} · $\lambda$-cl(q5) = {q5, q11} · $\lambda$-cl(q8) = {q8, q6, q9, q11} · $\lambda$-cl(q6) = {q6} · $\lambda$-cl(q7) = {q6, q7, q9, q11} · $\lambda$-cl(q9) = {q9, q11} · $\lambda$-cl(q11) = {q11}

**Lista 3** · **Ejercicio 248.** AFN-$\lambda$ de 10 estados; AFD mínimo de 5 estados.

$\lambda$-cl(q6) = {q6, q2, q0, q3, q4, q7, q8} · $\lambda$-cl(q2) = {q2, q0, q3, q7, q8} · $\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q0, q1, q3, q7, q8} · $\lambda$-cl(q3) = {q3, q7, q8} · $\lambda$-cl(q4) = {q4} · $\lambda$-cl(q5) = {q5, q7, q8} · $\lambda$-cl(q7) = {q7, q8} · $\lambda$-cl(q8) = {q8} · $\lambda$-cl(q9) = {q9}

**Lista 3** · **Ejercicio 249.** AFN-$\lambda$ de 12 estados; AFD mínimo de 3 estados.

$\lambda$-cl(q6) = {q6, q2, q0, q3, q4, q7, q10, q8, q11} · $\lambda$-cl(q2) = {q2, q0, q3, q7, q10, q8, q11} · $\lambda$-cl(q0) = {q0} · $\lambda$-cl(q1) = {q0, q1, q3, q7, q10, q8, q11} · $\lambda$-cl(q3) = {q3, q7, q10, q8, q11} · $\lambda$-cl(q4) = {q4} · $\lambda$-cl(q5) = {q5, q7, q10, q8, q11} · $\lambda$-cl(q7) = {q7, q10, q8, q11} · $\lambda$-cl(q10) = {q10, q8, q11} · $\lambda$-cl(q8) = {q8} · $\lambda$-cl(q9) = {q8, q9, q11} · $\lambda$-cl(q11) = {q11}

**Lista 3** · **Ejercicio 250.** AFN-$\lambda$ de 16 estados; AFD mínimo de 6 estados.

$\lambda$-cl(q14) = {q14, q2, q0, q3, q6, q4, q7, q10, q8, q11, q12, q15} · $\lambda$-cl(q2) = {q2, q0, q3, q6, q4, q7,

q15} · λ-cl(q0) = {q0} · λ-cl(q1) = {q0, q1, q3, q6, q4, q7, q15} · λ-cl(q3) = {q3, q6, q4, q7, q15} · λ-cl(q6) = {q6, q4, q7, q15} · λ-cl(q4) = {q4} · λ-cl(q5) = {q4, q5, q7, q15} · λ-cl(q7) = {q7, q15} · λ-cl(q10) = {q10, q8, q11, q12} · λ-cl(q8) = {q8} · λ-cl(q9) = {q8, q9, q11, q12} · λ-cl(q11) = {q11, q12} · λ-cl(q12) = {q12} · λ-cl(q13) = {q13, q15} · λ-cl(q15) = {q15}

**Lista 3** · **Ejercicio 251.** AFN-λ de 18 estados; AFD mínimo de 5 estados.

λ-cl(q16) = {q16, q6, q0, q4, q2, q5, q7, q8, q12, q10, q13, q14} · λ-cl(q6) = {q6, q0, q4, q2, q5, q7, q8} · λ-cl(q0) = {q0} · λ-cl(q1) = {q1, q7, q8} · λ-cl(q4) = {q4, q2, q5, q7, q8} · λ-cl(q2) = {q2} · λ-cl(q3) = {q2, q3, q5, q7, q8} · λ-cl(q5) = {q5, q7, q8} · λ-cl(q7) = {q7, q8} · λ-cl(q8) = {q8} · λ-cl(q9) = {q9, q17} · λ-cl(q12) = {q12, q10, q13, q14} · λ-cl(q10) = {q10} · λ-cl(q11) = {q10, q11, q13, q14} · λ-cl(q13) = {q13, q14} · λ-cl(q14) = {q14} · λ-cl(q15) = {q15, q17} · λ-cl(q17) = {q17}

**Lista 3** · **Ejercicio 252.** AFN-λ de 38 estados; AFD mínimo de 8 estados.

λ-cl(q36) = {q36, q6, q4, q0, q2, q7, q16, q8, q12, q26, q18, q22} · λ-cl(q6) = {q6, q4, q0, q2, q7, q16, q8, q12} · λ-cl(q4) = {q4, q0, q2} · λ-cl(q0) = {q0} · λ-cl(q1) = {q4, q0, q1, q2, q5, q7, q16, q8, q12} · λ-cl(q2) = {q2} · λ-cl(q3) = {q4, q0, q2, q3, q5, q7, q16, q8, q12} · λ-cl(q5) = {q4, q0, q2, q5, q7, q16, q8, q12} · λ-cl(q7) = {q7, q16, q8, q12} · λ-cl(q16) = {q16, q8, q12} · λ-cl(q8) = {q8} · λ-cl(q9) = {q9, q10} · λ-cl(q10) = {q10} · λ-cl(q11) = {q11, q17, q37} · λ-cl(q12) = {q12} · λ-cl(q13) = {q13, q14} · λ-cl(q14) = {q14} · λ-cl(q15) = {q15, q17, q37} · λ-cl(q17) = {q17, q37} · λ-cl(q26) = {q26, q18, q22} · λ-cl(q18) = {q18} · λ-cl(q19) = {q19, q20} · λ-cl(q20) = {q20} · λ-cl(q21) = {q21, q27, q34, q32, q28, q30, q35, q37} · λ-cl(q22) = {q22} · λ-cl(q23) = {q23, q24} · λ-cl(q24) = {q24} · λ-cl(q25) = {q25, q27, q34, q32, q28, q30, q35, q37} · λ-cl(q27) = {q27, q34, q32, q28, q30, q35, q37} · λ-cl(q34) = {q34, q32, q28, q30, q35, q37} · λ-cl(q32) = {q32, q28, q30} · λ-cl(q28) = {q28} · λ-cl(q29) = {q32, q28, q29, q30, q33, q35, q37} · λ-cl(q30) = {q30} · λ-cl(q31) = {q32, q28, q30, q31, q33, q35, q37} · λ-cl(q33) = {q32, q28, q30, q33, q35, q37} · λ-cl(q35) = {q35, q37} · λ-cl(q37) = {q37}

**Lista 3** · **Ejercicio 253.** AFN-λ de 44 estados; AFD mínimo de 18 estados.

λ-cl(q14) = {q14, q0, q12, q10, q2, q6, q13, q15, q32, q30, q24, q16, q20, q26, q33, q42, q34, q35, q36, q43} · λ-cl(q0) = {q0} · λ-cl(q1) = {q1, q15, q32, q30, q24, q16, q20, q26, q33, q42, q34, q35, q36, q43} · λ-cl(q12) = {q12, q10, q2, q6, q13, q15, q32, q30, q24, q16, q20, q26, q33, q42, q34, q35, q36, q43} · λ-cl(q10) = {q10, q2, q6} · λ-cl(q2) = {q2} · λ-cl(q3) = {q3, q4} · λ-cl(q4) = {q4} · λ-cl(q5) = {q10, q2, q5, q6, q11, q13, q15, q32, q30, q24, q16, q20, q26, q33, q42, q34, q35, q36, q43} · λ-cl(q6) = {q6} · λ-cl(q7) = {q7, q8} · λ-cl(q8) = {q8} · λ-cl(q9) = {q10, q2, q6, q9, q11, q13, q15, q32, q30, q24, q16, q20, q26, q33, q42, q34, q35, q36, q43} · λ-cl(q11) = {q10, q2, q6, q11, q13, q15, q32, q30, q24, q16, q20, q26, q33, q42, q34, q35, q36, q43} · λ-cl(q13) = {q13, q15, q32, q30, q24, q16, q20, q26, q33, q42, q34, q35, q36, q43} · λ-cl(q15) = {q15, q32, q30, q24, q16, q20, q26, q33, q42, q34, q35, q36, q43} · λ-cl(q32) = {q32, q30, q24, q16, q20, q26, q33, q42, q34, q35, q36, q43} · λ-cl(q30) = {q30, q24, q16, q20, q26} · λ-cl(q24) = {q24, q16, q20} · λ-cl(q16) = {q16} · λ-cl(q17) = {q17, q18} · λ-cl(q18) = {q18} · λ-cl(q19) = {q30, q24, q16, q19, q20, q25, q26, q31, q33, q42, q34, q35, q36, q43} · λ-cl(q20) = {q20} · λ-cl(q21) = {q21, q22} · λ-cl(q22) = {q22} · λ-cl(q23) = {q30, q24, q16, q20, q23, q25, q26, q31, q33, q42, q34, q35, q36, q43} · λ-cl(q25) = {q30, q24, q16, q20, q25, q26, q31, q33, q42, q34, q35, q36, q43} · λ-cl(q26) = {q26} · λ-cl(q27) = {q27, q28} · λ-cl(q28) = {q28} · λ-cl(q29) = {q30, q24, q16, q20, q26, q29, q31, q33, q42, q34, q35, q36, q43} · λ-cl(q31) = {q30, q24, q16, q20, q26, q31, q33, q42, q34, q35, q36, q43} · λ-cl(q33) = {q33, q42, q34, q35, q36, q43} · λ-cl(q42) = {q42, q34, q35, q36, q43} · λ-cl(q34) = {q34, q35, q43} · λ-cl(q35) = {q35, q43} · λ-cl(q36) = {q36} · λ-cl(q37) = {q37, q38} · λ-cl(q38) = {q38} · λ-cl(q39) = {q39, q40} · λ-cl(q40) = {q40} · λ-cl(q41) = {q41, q43} · λ-cl(q43) = {q43}

# C. Anexo C. Conversiones

**Tabla 13.** Tamaño de los autómatas antes y después de convertir.

| Ejercicio | Tipo | Estados del AFN | Estados del AFD | Estados del AFD mínimo | Cota $2^n$ |
|---|---|---|---|---|---|
| Lista 3 · AFN del problema 094 | AFN | 3 | 3 | 3 | 8 |
| Lista 3 · AFN del problema 098 | AFN | 6 | 6 | 6 | 64 |
| Lista 3 · AFN del problema 099 | AFN | 7 | 7 | 7 | 128 |
| Lista 3 · AFN del problema 101 | AFN | 4 | 6 | 4 | 16 |
| Lista 3 · AFN del problema 102 | AFN | 7 | 12 | 7 | 128 |
| Lista 3 · AFN del problema 103 | AFN | 7 | 12 | 7 | 128 |
| Lista 3 · AFN del problema 105 | AFN | 5 | 5 | 5 | 32 |
| Lista 3 · AFN del problema 106 | AFN | 8 | 14 | 8 | 256 |
| Lista 3 · AFN del problema 107 | AFN | 16 | 20 | 13 | 65536 |
| Lista 3 · AFN del problema 108 | AFN | 23 | 88 | 37 | 8388608 |
| Lista 3 · AFN del problema 109 | AFN | 26 | 40 | 22 | 67108864 |
| Lista 3 · AFN del problema 112 | AFN | 7 | 11 | 7 | 128 |
| Lista 3 · AFN del problema 113 | AFN | 11 | 13 | 7 | 2048 |

| Ejercicio | Tipo | Estados del AFN | Estados del AFD | Estados del AFD mínimo | Cota $2^n$ |
|---|---|---|---|---|---|
| Lista 3 · AFN del problema 118 | AFN | 5 | 5 | 5 | 32 |
| Lista 3 · AFN del problema 120 | AFN | 5 | 7 | 7 | 32 |
| Lista 3 · AFN del problema 123 | AFN | 14 | 15 | 15 | 16384 |
| Lista 3 · Ejercicio 189 | AFN | 7 | 12 | 7 | 128 |
| Lista 3 · Ejercicio 190 | AFN | 26 | 32 | 19 | 67108864 |
| Lista 3 · Ejercicio 192 | AFN | 3 | 5 | 5 | 8 |
| Lista 3 · Ejercicio 193 | AFN | 4 | 7 | 6 | 16 |
| Lista 3 · Ejercicio 194 | AFN | 4 | 4 | 2 | 16 |
| Lista 3 · Ejercicio 195 | AFN | 5 | 12 | 7 | 32 |
| Lista 3 · Ejercicio 196 | AFN-$\lambda$ | 3 | 3 | 2 | 8 |
| Lista 3 · Ejercicio 197 | AFN-$\lambda$ | 4 | 2 | 1 | 16 |
| Lista 3 · Ejercicio 198 | AFN-$\lambda$ | 4 | 3 | 2 | 16 |
| Lista 3 · Ejercicio 199 | AFN-$\lambda$ | 5 | 4 | 2 | 32 |
| Lista 3 · Ejercicio 200 | AFN-$\lambda$ | 6 | 6 | 2 | 64 |
| Lista 3 · Ejercicio 236 | AFN-$\lambda$ | 22 | 9 | 4 | 4194304 |
| Lista 3 · Ejercicio 237 | AFN-$\lambda$ | 14 | 5 | 4 | 16384 |
| Lista 3 · Ejercicio 238 | AFN-$\lambda$ | 34 | 11 | 6 | 17179869184 |
| Lista 3 · Ejercicio 239 | AFN-$\lambda$ | 26 | 9 | 6 | 67108864 |

| Ejercicio | Tipo | Estados del AFN | Estados del AFD | Estados del AFD mínimo | Cota $2^n$ |
|---|---|---|---|---|---|
| Lista 3 · Ejercicio 240 | AFN-$\lambda$ | 78 | 27 | 6 | 302231454903657293676544 |
| Lista 3 · Ejercicio 241 | AFN-$\lambda$ | 6 | 3 | 2 | 64 |
| Lista 3 · Ejercicio 242 | AFN-$\lambda$ | 8 | 3 | 3 | 256 |
| Lista 3 · Ejercicio 243 | AFN-$\lambda$ | 8 | 3 | 1 | 256 |
| Lista 3 · Ejercicio 244 | AFN-$\lambda$ | 8 | 4 | 3 | 256 |
| Lista 3 · Ejercicio 245 | AFN-$\lambda$ | 10 | 4 | 2 | 1024 |
| Lista 3 · Ejercicio 246 | AFN-$\lambda$ | 10 | 3 | 3 | 1024 |
| Lista 3 · Ejercicio 247 | AFN-$\lambda$ | 12 | 4 | 4 | 4096 |
| Lista 3 · Ejercicio 248 | AFN-$\lambda$ | 10 | 4 | 4 | 1024 |
| Lista 3 · Ejercicio 249 | AFN-$\lambda$ | 12 | 4 | 2 | 4096 |
| Lista 3 · Ejercicio 250 | AFN-$\lambda$ | 16 | 6 | 5 | 65536 |
| Lista 3 · Ejercicio 251 | AFN-$\lambda$ | 18 | 4 | 4 | 262144 |
| Lista 3 · Ejercicio 252 | AFN-$\lambda$ | 38 | 13 | 8 | 274877906944 |
| Lista 3 · Ejercicio 253 | AFN-$\lambda$ | 44 | 22 | 17 | 17592186044416 |
| Lista 3 · Ejercicio 260 | AFN | 5 | 7 | 6 | 32 |
| Lista 3 · Ejercicio 261 | AFN | 5 | 5 | 3 | 32 |
| Lista 3 · Ejercicio 263 | AFN | 3 | 3 | 3 | 8 |